Journal of Computational and Applied Mathematics 451 (2024) 116044

Contents lists available at ScienceDirect

Journal of Computational and Applied Mathematics

journal homepage: www.elsevier.com/locate/cam

Randomized compression of rank-structured matrices accelerated with graph coloring

James Levitt, Per-Gunnar Martinsson *

Oden Institute for Computational Engineering and Sciences, University of Texas at Austin, Austin, TX, United States of America

ARTICLE INFO

MSC:
65N22
65N38
15A23
15A52

Keywords:
Randomized approximation of matrices
Rank-structured matrices
HODLR matrix
Hierarchically block separable matrix
Hierarchically semiseparable matrix
Fast direct solver

ABSTRACT

A randomized algorithm for computing a data sparse representation of a given rank-structured matrix $\mathbf{A}$ (a.k.a. an $\mathcal{H}$-matrix) is presented. The algorithm draws on the randomized singular value decomposition (RSVD), and operates under the assumption that methods for rapidly applying $\mathbf{A}$ and $\mathbf{A}^*$ to vectors are available. The algorithm uses graph coloring algorithms to analyze the hierarchical tree that defines the rank structure to generate a tailored probability distribution from which to draw the random test matrices. The matrix is then applied to the test matrices, and in a final step the matrix itself is reconstructed by the observed input–output pairs. The method presented is an evolution of the "peeling algorithm" of Lin et al. (2011). For the case of uniform trees, the new method substantially reduces the pre-factor of the original peeling algorithm. More significantly, the new technique leads to dramatic acceleration for many non-uniform trees since it constructs test matrices that are optimized for a given tree. The algorithm is particularly effective for kernel matrices involving a set of points restricted to a lower dimensional object than the ambient space, such as a boundary integral equation defined on a surface in three dimensions.

1. Introduction

We present a set of efficient algorithms for computing data-sparse representations of large dense matrices that have *rank structure*. This property is often defined to mean that an $N \times N$ matrix can be tessellated into $O(N)$ blocks in such a way that each block is either small or of low numerical rank, cf. Figs. 1 and 3. Matrices with such structure can be stored and applied to vectors efficiently, often with cost that scales linearly or close to linearly with N. It is in many environments also possible to compute an approximate inverse or LU factorization in linear or close to linear time. The notion of rank-structured matrices appears implicitly in techniques such as Barnes–Hut [1] and the Fast Multipole Method [2]. The idea was then brought to the forefront as a linear algebraic structure by Hackbusch and co-workers [3,4], and is now a subject of research under names such as $\mathcal{H}$-matrices [3,5,6]; HODLR matrices [7,8], Hierarchically Semi-Separable (HSS) a.k.a. Hierarchically Block-Separable (HBS) matrices [9–12], Recursive Skeletonization [13–15], and many more.

The question we address is the following: Suppose that you are given an $N \times N$ matrix $\mathbf{A}$ that you know is rank structured under some given partitioning, but you do not have direct access to the low-rank factors that define the compressible off-diagonal blocks. Instead, you have access to some efficient procedure that, given tall thin matrices $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times \ell}$, can evaluate the matrix–matrix products

$$\mathbf{Y} = \mathbf{A}\mathbf{\Omega}, \qquad \text{and} \qquad \mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}.$$

* Corresponding author.
E-mail addresses: jlevitt@utexas.edu (J. Levitt), pgm@oden.utexas.edu (P.-G. Martinsson).

https://doi.org/10.1016/j.cam.2024.116044
Received 9 July 2023; Received in revised form 6 March 2024
Available online 12 June 2024

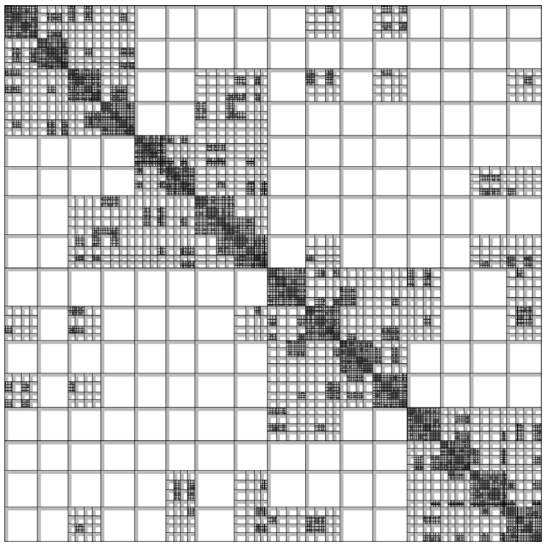


Fig. 1. An $\mathcal{H}^1$ matrix for a quadtree over a uniform grid in the plane. Dense blocks are shown in dark gray, and low-rank blocks are represented with a white background and light gray rectangles representing the shapes of the low-rank factors.

Your task is to build two random (but structured) matrices $\mathbf{\Omega}$ and $\mathbf{\Psi}$ that have the property that $\mathbf{A}$ can be recovered from the information in the set $\{\mathbf{Y}, \mathbf{\Omega}, \mathbf{Z}, \mathbf{\Psi}\}$ in a computationally efficient manner. The algorithms described here solve the reconstruction problem using $\ell \sim k \log(N)$ sample vectors, where k is an upper bound on the ranks of the off-diagonal blocks. The key novelty is the formulation of a graph coloring problem to analyze the cluster tree that defines the rank structure to build test matrices that are optimized for the specific problem under consideration. (We assume that the correct partition is already known, typically from the geometry of an underlying physical problem.)

The method that we describe relies on the user having access to an efficient algorithm for applying the rank-structured matrix. An important application of such a method concerns the compression of the dense Schur complements that arise in the LU factorization of sparse matrices associated with finite difference and finite element discretizations of elliptic PDEs. [11,16–18]. During the LU factorization, the user needs to form a data-sparse representation of a Schur complement such as $\mathbf{S}_{22} = \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}$, representing the elimination of the diagonal block $\mathbf{A}_{11}$. Commonly, the matrices $\mathbf{A}_{21}$, $\mathbf{A}_{11}$, $\mathbf{A}_{12}$ are already held in a sparse or data-sparse form, which means that the map $\mathbf{x} \mapsto \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}\mathbf{x}$ can be evaluated rapidly. In contrast, explicitly forming $\mathbf{A}_{11}^{-1}$ and the matrix products is much more challenging. Another application of our technique concerns the multiplication of dense operators in potential theory, where each factor operator can be applied using a legacy technique such as the FMM [2], and where inverses can often be applied via "fast direct solvers" [19].

The method we describe is inspired by the "peeling algorithm" of [20], which to the best of our knowledge was the first true black-box algorithm described in the literature. The method of [20] has the same asymptotic sample complexity $\ell \sim k \log(N)$ as our method, but involves substantially larger pre-factors. To be precise, [20] is targeted specifically for $\mathcal{H}^1$- and $\mathcal{H}^2$-matrices arising from the discretization of integral equations. Strong admissibility, and regular tree structures are used. In this environment, the method requires $\ell \sim k\, 8^d \log(N)$ matrix–vector products involving $\mathbf{A}$ and $\mathbf{A}^*$, where d is the dimension of space in which the underlying integral equation is defined. In contrast, the method presented here has complexity $\ell \sim k\, 6^d \log(N)$ for fully populated uniform trees. For more general trees, the acceleration over the method of [20] can be even more dramatic, since the adaptivity of our method enables it to "discover" intrinsic low dimensional structure in a given problem. As an illustration, Section 5 reports on experiments involving a boundary integral equation defined on a 2D surface in three dimensional space, as well as examples in higher dimensions.

Another advantage of the techniques presented here is that they are not limited to the $\mathcal{H}^1$ structure. To compress uniform $\mathcal{H}^1$ and $\mathcal{H}^2$ matrices, the presented algorithms obtain uniform basis matrices (cf. [3, Sec. 4.6]) by sampling the interactions of a box with its entire interaction list collectively. This process results in higher quality samples while requiring fewer matrix–vector products compared to existing methods that approximate interactions between boxes separately and then apply a recompression step to obtain uniform basis matrices. More generally, the formulation of a graph coloring problem can be used to design test matrices for any tessellation (e.g., arising from some other geometric or algebraic admissibility condition).

Remark 1.1 (*Related work*)**.** A kindred class of algorithms for computing a rank-structured matrix by observing its action on random vectors was described in [21,22]. These techniques have actual linear complexity (no logarithmic terms), and tend to be very fast in practice. However, they are not true black-box algorithms, as they require the direct evaluation of a small number of entries of the matrix. In contrast, the method presented here is truly black box, like the methods of [20,23].

The recent work [24] describes an algorithm for compressing Hierarchically Block Separable (HBS) matrices (also known as Hierarchically Semiseparable or HSS matrices), which is truly black box and has linear complexity. However, that work relies on the assumption that the basis matrices are nested, and it only considers the HBS format, which is subject to a weak admissibility condition. The present work focuses on rank-structured matrices subject to strong admissibility, including the $\mathcal{H}^1$ and uniform $\mathcal{H}^1$ formats, which do not assume nested basis matrices.

The paper [25] explores the problem of sparse direct solvers for discretized elliptic operators using techniques related to those presented here. Specifically, [25] relies on graph coloring to analyze the sparsity graph associated with a discretized PDE, and uses a "peeling algorithm" to build an approximate, sparsity preserving factorization.

The manuscript is structured as follows: Section 2 surveys some basic linear algebraic techniques that we rely on. Section 3 introduces our formalism for rank-structured matrices. Section 4 describes the new algorithm, and analyzes its asymptotic complexity. Section 5 describes numerical results.

2. Preliminaries

2.1. Notation

Throughout the paper, we measure a vector $\mathbf{x} \in \mathbb{R}^n$ by its Euclidean norm $\|\mathbf{x}\| = \left(\sum_i |x_i|^2\right)^{\frac{1}{2}}$. We measure a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ with the corresponding operator norm $\|\mathbf{A}\| = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$, and in some cases with the Frobenius norm $\|\mathbf{A}\|_{\text{Fro}} = (\sum_{i,j} |\mathbf{A}(i,j)|^2)^{1/2}$. To denote submatrices, we use the notation of Golub and Van Loan [26]: If $\mathbf{A}$ is an $m \times n$ matrix, and $I = [i_1, i_2, \ldots, i_k]$ and $J = [j_1, j_2, \ldots, j_l]$, then $\mathbf{A}(I, J)$ denotes the $k \times l$ submatrix of $\mathbf{A}$ at the intersection of the rows in I and the columns in J. We let $\mathbf{A}(I, :)$ denote the submatrix $\mathbf{A}(I, [1, 2, \ldots, n])$ and define $\mathbf{A}(:, J)$ analogously. We define the complement I^c of row index set I to be the set of row indices of $\mathbf{A}$ not included in I, $I^c = \{1, 2, \ldots, m\} \setminus I$, and we define the complement of a column index set analogously. We let $\mathbf{A}^*$ denote the transpose of $\mathbf{A}$, and we say that matrix $\mathbf{U}$ is *orthonormal* if its columns are orthonormal, $\mathbf{U}^*\mathbf{U} = \mathbf{I}$.

2.2. The QR factorization

The column-pivoted *QR factorization* of a matrix $\mathbf{A}$ of size $m \times n$ takes the form

$$\underset{m\times n}{\mathbf{A}} \quad \underset{n\times n}{\mathbf{P}} = \underset{m\times r}{\mathbf{Q}} \quad \underset{r\times n}{\mathbf{R}}, \tag{2.1}$$

where $r = \min(m, n)$, $\mathbf{Q}$ is orthonormal, $\mathbf{R}$ is upper-triangular, and $\mathbf{P}$ is a permutation matrix. Representing the permutation matrix $\mathbf{P}$ as the vector $J \subset \mathbb{Z}_+^n$ of column indices such that $\mathbf{P} = \mathbf{I}(:, J)$, the factorization Eq. (2.1) can be expressed as

$$\underset{m\times n}{\mathbf{A}(:,J)} = \underset{m\times r}{\mathbf{Q}} \quad \underset{r\times n}{\mathbf{R}}.$$

For a matrix that is numerically low-rank, a rank-k approximation of $\mathbf{A}$ is given by a "partial QR factorization of $\mathbf{A}$",

$$\underset{m\times n}{\mathbf{A}(:,J)} \approx \underset{m\times k}{\mathbf{Q}_k} \quad \underset{k\times n}{\mathbf{R}_k}.$$

2.3. The singular value decomposition (SVD)

The singular value decomposition of a matrix $\mathbf{A}$ of size $m \times n$ takes the form

$$\underset{m\times n}{\mathbf{A}} = \underset{m\times r}{\mathbf{U}} \quad \underset{r\times r}{\mathbf{\Sigma}} \quad \underset{r\times n}{\mathbf{V}^*}, \tag{2.2}$$

where $r = \min(m, n)$, $\mathbf{U}$ and $\mathbf{V}$ are orthonormal matrices, and $\mathbf{\Sigma}$ is a diagonal matrix with diagonal elements $\{\sigma_j\}_{j=1}^r$ ordered such that $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$. The columns of $\mathbf{U}$ and $\mathbf{V}$, denoted by $\{\mathbf{u}_i\}_{i=1}^r$ and $\{\mathbf{v}_i\}_{i=1}^r$, are the left and right singular vectors of $\mathbf{A}$. We let $\mathbf{A}_k$ denote the rank-k approximation obtained by truncating the SVD to its first k terms, so that $\mathbf{A}_k = \sum_{j=1}^k \sigma_j \mathbf{u}_j \mathbf{v}_j^*$. It then follows that

$$\|\mathbf{A} - \mathbf{A}_k\| = \sigma_{k+1} \qquad \text{and that} \qquad \|\mathbf{A} - \mathbf{A}_k\|_{\text{Fro}} = \left(\sum_{j=k+1}^{\min(m,n)} \sigma_j^2\right)^{1/2}.$$

The Eckart–Young theorem asserts that $\mathbf{A}_k$ achieves the smallest possible approximation error of $\mathbf{A}$, in both the operator and Frobenius norms, of any rank-k matrix.

2.4. Functions for low-rank factorizations

We introduce the following notation to denote calls to functions that return the results of QR factorizations and singular value decompositions. Function calls to evaluate the full factorizations are written as

$$[\mathbf{Q}, \mathbf{R}, J] = \texttt{qr}(\mathbf{A}), \qquad [\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}] = \texttt{svd}(\mathbf{A}).$$

Calls to evaluate the rank-k truncated factorizations are written as

$$[\mathbf{Q}, \mathbf{R}, J] = \texttt{qr}(\mathbf{A}, k), \qquad [\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}] = \texttt{svd}(\mathbf{A}, k).$$

We write

$$[\mathbf{Q}, \mathbf{R}] = \texttt{qr}(\mathbf{A}), \qquad \mathbf{Q} = \texttt{qr}(\mathbf{A})$$

to compute an unpivoted QR factorization, and just the factor $\mathbf{Q}$, respectively.

2.5. Randomized compression

In this section, we give a brief review of randomized low-rank approximation, following the presentation of [23]. Let $\mathbf{A}$ be an $m \times n$ matrix that can be accurately approximated by a matrix of rank k, and suppose we seek to determine a matrix $\mathbf{Q}$ with orthonormal columns (as few as possible) such that $\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^*\mathbf{A}\|$ is small. In other words, we seek a matrix $\mathbf{Q}$ whose columns form an approximate orthornomal basis (ON-basis) for the column space of $\mathbf{A}$. This task can efficiently be solved via the following randomized procedure:

1. Pick a small integer p representing how much "oversampling" is done. ($p = 10$ is often good.)
2. Form an $n \times (k + p)$ matrix $\mathbf{G}$ whose entries are drawn independently from a standardized normal ("Gaussian") distribution.
3. Form the "sample matrix" $\mathbf{Y} = \mathbf{A}\mathbf{G}$ of size $m \times (k + p)$.
4. Construct an $m \times (k + p)$ matrix $\mathbf{Q}$ whose columns form an ON basis for the columns of $\mathbf{Y}$.

Note that each column of the sample matrix $\mathbf{Y}$ is a random linear combination of the columns of $\mathbf{A}$. The probability of the algorithm producing an accurate result approaches 1 extremely rapidly as p increases, and, remarkably, this probability depends only on p (not on m or n, or any other properties of $\mathbf{A}$); cf. [27].

Now, suppose we would like to use the above sampling procedure to compute a low-rank factorization of the form

$$\underset{m \times n}{\mathbf{A}} = \underset{m \times r}{\mathbf{U}} \quad \underset{r \times r}{\mathbf{B}} \quad \underset{r \times n}{\mathbf{V}}, \tag{2.3}$$

where $\mathbf{U}$ and $\mathbf{V}$ have orthonormal columns and $\mathbf{B}$ is a square matrix, not necessarily diagonal. We review two methods of completing this task using randomized sampling. The algorithm summarized in Algorithm 2.1 involves two randomized samples, one of $\mathbf{A}$ and one of $\mathbf{A}^*$. It is based on so-called "single-view" algorithms for matrix compression [28,29]. The second method [27], summarized in Algorithm 2.2, uses a randomized sample of $\mathbf{A}$ to compute the column basis matrix $\mathbf{U}$, and then operates on the product $\mathbf{A}^*\mathbf{U}$ to obtain $\mathbf{B}$ and $\mathbf{V}$. We will use both methods in the algorithms described in Section 4. (Though it is sometimes prudent to do slightly more over-sampling in the "single view" environment [28,29], we use the same p in both cases.)

Algorithm 2.1 Compress $\mathbf{A} \approx \mathbf{U}\mathbf{B}\mathbf{V}^*$ via two randomized samples

Form $n \times (k + p)$ Gaussian random matrix $\mathbf{G}_1$.
Multiply $\mathbf{Y} = \mathbf{A}\mathbf{G}_1$.
Orthonormalize $\mathbf{U} = \texttt{qr}(\mathbf{Y}, k)$.
Form $m \times (k + p)$ Gaussian random matrix $\mathbf{G}_2$.
Multiply $\mathbf{Z} = \mathbf{A}^*\mathbf{G}_2$.
Orthonormalize $\mathbf{V} = \texttt{qr}(\mathbf{Z}, k)$.
Solve $\mathbf{B} = (\mathbf{G}_2^*\mathbf{U})^\dagger \mathbf{G}_2^*\mathbf{A}\mathbf{G}_1(\mathbf{V}^*\mathbf{G}_1)^\dagger$.
return $\mathbf{U}, \mathbf{B}, \mathbf{V}$.

Algorithm 2.2 Compress $\mathbf{A} \approx \mathbf{U}\mathbf{B}\mathbf{V}^*$ with one randomized and one deterministic sample

Form an $n \times (k + p)$ Gaussian random matrix $\mathbf{G}$.
Multiply $\mathbf{Y} = \mathbf{A}\mathbf{G}$.
Orthonormalize $\mathbf{U} = \texttt{qr}(\mathbf{Y})$.
Multiply $\mathbf{W} = \mathbf{A}^*\mathbf{U}$.
Orthonormalize $[\mathbf{V}, \mathbf{B}^*] = \texttt{qr}(\mathbf{W})$.
return $\mathbf{U}, \mathbf{B}, \mathbf{V}$.

2.6. The degree of saturation graph coloring algorithm

A vertex coloring of a graph is an assignment of a color to each vertex in such a way that no pair of adjacent vertices shares the same color. The problem of finding a vertex coloring with the minimum number of colors is NP-hard, but there exist a number of algorithms for efficiently coloring graphs, though they may produce colorings with more than the minimum number of colors.

Greedy graph coloring algorithms process the vertices in sequence, at each iteration assigning to one vertex the first available color that has not already been assigned to one of its neighbors. In the *degree of saturation* (DSatur) algorithm [30], the choice of which vertex to color at each step is made by selecting from the remaining uncolored vertices the one whose neighbors have the greatest number of distinct colors (the so-called degree of saturation). The procedure is summarized in Algorithm 2.3.

Algorithm 2.3 Greedy graph coloring with DSatur

```
Initialize priority queue q of vertices keyed by degree of saturation (initially all zero)
Initialize for each vertex a set of invalid colors (initially all empty)
while q is not empty do
    Pop from q the vertex v with highest degree of saturation          ▷ O(log |V|)
    Assign a color to v, creating a new one if necessary               ▷ O(deg(G))
    for each vertex w adjacent to v do
        Add the color of v to the set of invalid colors for w          ▷ O(1)
        Update the priority of w within q                              ▷ O(log |V|)
```

While the asymptotic complexity of DSatur is often described as quadratic in the number of vertices, the algorithm can be implemented with quasilinear complexity, assuming the degree of the graph is bounded. The *degree of a vertex* is the number of edges incident to that vertex, and the *degree of a graph* is the maximum degree over all of its vertices. Summing the costs listed in Algorithm 2.3, we find that the asymptotic complexity is

$$T_{\text{color}} \sim \deg(G)|V| \log |V|, \tag{2.4}$$

where $\deg(G)$ denotes the degree of the graph. For the step of assigning a color to vertex v, we note that a greedy coloring algorithm uses no more than $\deg(G)+1$ colors in the worst case, so checking the existing colors for whether they belong to the set of invalid colors for v requires $\mathcal{O}(\deg(G))$ operations.

3. Rank-structured matrices

This section provides definitions of the rank-structured matrix formats that we use, following the notational framework of [19]. Let $\{x_i\}_{i=1}^N \subset X$ be a set of points within a d-dimensional hypercube $X = [0,1]^d$, and let $\mathbf{A}$ be an N-by-N matrix where

$$\mathbf{A}(i,j) = \mathcal{K}(x_i, x_j) \qquad 1 \le i \le N, 1 \le j \le N,$$

for some kernel function $\mathcal{K}$. That is, the (i,j) entry of $\mathbf{A}$ represents an interaction between points x_i, x_j as defined by $\mathcal{K}$.

We divide the domain into a number of boxes and arrange the boxes into a tree structure, where the levels of the tree represent successively finer partitions of X. Level 0 only contains a single box, which encloses all of X. The boxes belonging to level $l+1$ are obtained by bisecting each of the boxes in level l along each dimension to form 2^d smaller boxes. The 2^d boxes in level $l+1$ obtained by splitting a box in level l are designated as the children of that box, giving rise to the tree structure. Boxes that do not contain any points are omitted from the tree, so the branching factor of the tree may be less than 2^d, depending on the distribution of points. The splitting procedure is applied recursively to boxes that contain more than m points, where m is a prespecified maximum number of points to be allowed in a leaf box. We let L denote the depth of the tree, and assume that the points are distributed evenly enough across the domain so that $L \sim \log N$. For each box τ, we define $I_\tau = \{i : x_i \in \tau\}$ to be the set of indices of the points contained by τ.

Two boxes that belong to the same level and have overlapping boundaries are said to be *neighbors*. The neighbors of box τ, of which there are up to 3^d (including τ itself), are stored in the *neighbor list* of τ, denoted by $\mathcal{L}_\tau^{\text{nei}}$. The *interaction list* of τ, denoted by $\mathcal{L}_\tau^{\text{int}}$ contains the children of neighbors of the parent of τ, excluding those that are neighbors of τ. The interaction list of a box contains up to $6^d - 3^d$ boxes. Fig. 2 shows an example tree structure and index lists.

Matrix $\mathbf{A}$ is said to have $\mathcal{H}^1$ structure if block $\mathbf{A}(I_\alpha, I_\beta)$ is numerically low-rank, for all pairs of boxes α, β belonging to the interaction lists of one another. Such blocks, and the corresponding pairs of boxes, are said to be *admissible*. Fig. 3 shows a tessellation of a matrix consisting of the admissible blocks as well as a number of *inadmissible* blocks of interactions between neighboring boxes in level L, which are not necessarily low-rank. Constructing a compressed representation of an $\mathcal{H}^1$ matrix consists of finding low rank approximations

$$\mathbf{A}(I_\alpha, I_\beta) \approx \mathcal{U}_{\alpha,\beta} \mathbf{B}_{\alpha,\beta} \mathcal{V}_{\alpha,\beta}$$

for each admissible pair (α, β) and storing the inadmissible blocks corresponding to neighbor interactions of boxes in level L.

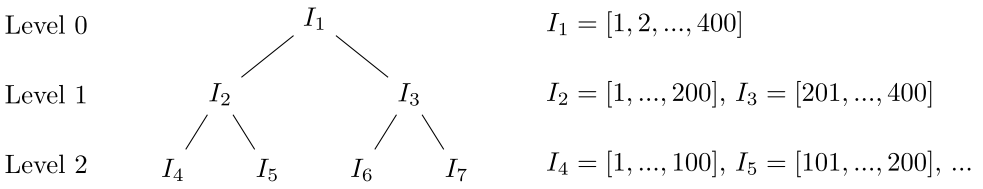


Fig. 2. A binary tree structure, where the levels of the tree represent successively refined partitions of the index vector $[1, \dots, 400]$.

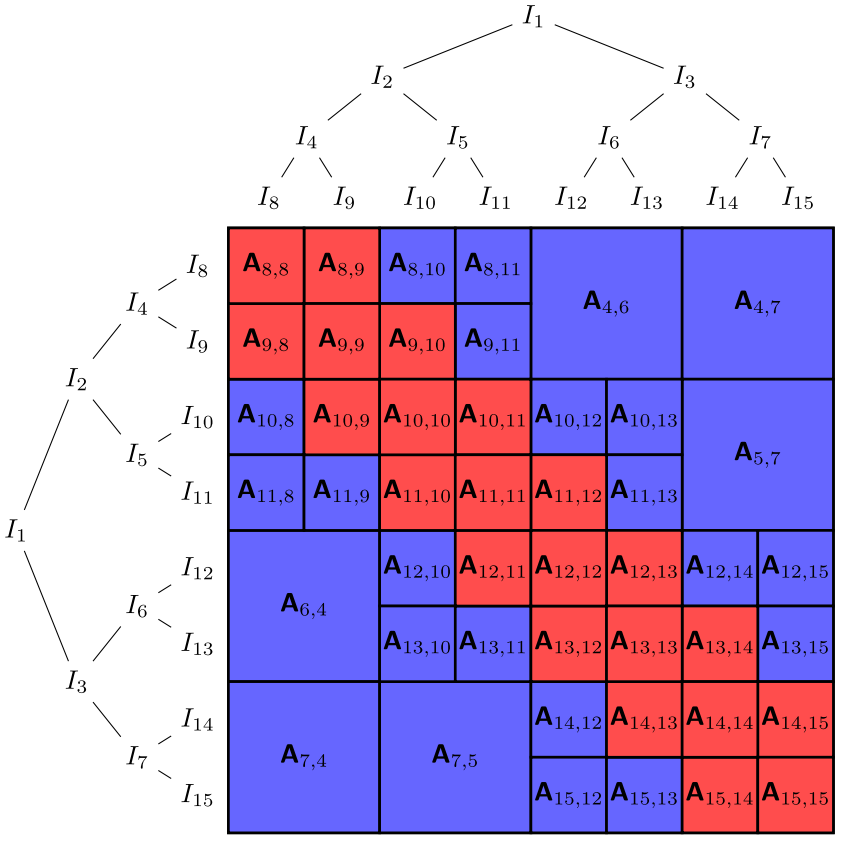


Fig. 3. An $\mathcal{H}^1$ matrix with depth 3 based on a grid over $[0, 1]$. Admissible blocks are shown in blue, and inadmissible blocks are shown in red.

An $\mathcal{H}^1$ matrix $\mathbf{A}$ is said to have uniform $\mathcal{H}^1$ structure if it further satisfies the condition that there exist low-rank basis matrices $\boldsymbol{\mathcal{U}}_\alpha$ spanning $\mathbf{A}\left(I_\alpha, \cup_{\beta \in \mathcal{L}_\alpha^{\text{int}}} I_\beta\right)$ and $\boldsymbol{\mathcal{V}}_\alpha$ spanning $\mathbf{A}\left(\cup_{\beta \in \mathcal{L}_\alpha^{\text{int}}} I_\beta, I_\alpha\right)$ for each box α. Constructing a compressed representation of a uniform $\mathcal{H}^1$ matrix consists of finding the matrices $\boldsymbol{\mathcal{U}}_\alpha, \boldsymbol{\mathcal{V}}_\alpha$ for each box α and the matrices $\mathbf{B}_{\alpha,\beta}$ such that

$$\mathbf{A}(I_\alpha, I_\beta) \approx \boldsymbol{\mathcal{U}}_\alpha \mathbf{B}_{\alpha,\beta} \boldsymbol{\mathcal{V}}_\beta$$

for each admissible pair (α, β) and storing the inadmissible blocks corresponding to neighbor interactions of boxes in level L.

A uniform $\mathcal{H}^1$ matrix $\mathbf{A}$ is said to have $\mathcal{H}^2$ structure if there exist low-rank basis matrices $\boldsymbol{\mathcal{U}}_\alpha$ spanning $\mathbf{A}\left(I_\alpha, \left(\cup_{\beta \in \mathcal{L}_\alpha^{\text{nei}}} I_\beta\right)^c\right)$ and $\boldsymbol{\mathcal{V}}_\alpha$ spanning $\mathbf{A}\left(\left(\cup_{\beta \in \mathcal{L}_\alpha^{\text{nei}}} I_\beta\right)^c, I_\alpha\right)$ for each box α. Then the basis matrices of a non-leaf box can be expressed in terms of the basis matrices of its children. For example, if τ is a box with children α, β, then

$$\boldsymbol{\mathcal{U}}_\tau = \begin{bmatrix} \boldsymbol{\mathcal{U}}_\alpha & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\mathcal{U}}_\beta \end{bmatrix} \mathbf{U}_\tau, \tag{3.1}$$

where $\boldsymbol{\mathcal{U}}_\gamma$ is the "long" column basis matrix of size $|\gamma| \times k$ associated with $\gamma \in \{\tau, \alpha, \beta\}$, and $\mathbf{U}_\tau$ is a "short" basis matrix of size $2k \times k$. Analogous relationships must hold for row basis matrices $\boldsymbol{\mathcal{V}}_\tau, \boldsymbol{\mathcal{V}}_\alpha, \boldsymbol{\mathcal{V}}_\beta$. Using such nested basis matrices eliminates the need to explicitly store $\boldsymbol{\mathcal{U}}_\tau$ since it is fully specified by $\boldsymbol{\mathcal{U}}_\alpha, \boldsymbol{\mathcal{U}}_\beta$ and $\mathbf{U}_\tau$. Likewise, if α or β have children of their own, then their basis matrices will be expressed in terms of their own small basis matrices and the basis matrices of their children.

4. Compressing rank-structured matrices with graph coloring

4.1. $\mathcal{H}^1$ H1 matrix compression

In this section, we present the process of constructing an $\mathcal{H}^1$ representation of a matrix by applying Algorithm 2.1 to compute low-rank approximations of the admissible blocks of levels $2, \dots, L$ and then extracting the inadmissible blocks of level L. We apply $\mathbf{A}$ and $\mathbf{A}^*$ to a set of carefully constructed test matrices, and from those products we extract randomized samples of each admissible block. We demonstrate the techniques on a matrix shown in Fig. 3, which is based on points in one dimension, but the algorithm generalizes in a straightforward way to points in higher dimensions.

4.1.1. Compressing level 2

We construct the compressed representation by processing levels of the tree in sequence from the coarsest level to the finest. There are no admissible blocks associated with levels 0 and 1, so we begin by computing low-rank approximations of the 6 admissible blocks associated with level 2 of the tree. To that end, we define four test matrices of size $N \times r$

$$\boldsymbol{\Omega}_1 = \begin{bmatrix} \mathbf{G}_4 \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \qquad \boldsymbol{\Omega}_2 = \begin{bmatrix} \mathbf{0} \\ \mathbf{G}_5 \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \qquad \boldsymbol{\Omega}_3 = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{G}_6 \\ \mathbf{0} \end{bmatrix}, \qquad \boldsymbol{\Omega}_4 = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{G}_7 \end{bmatrix},$$

where $\mathbf{G}_4, \mathbf{G}_5, \mathbf{G}_6, \mathbf{G}_7$ are random matrices of size $N/4 \times r$ whose entries are drawn from the standard normal distribution. We invoke the black-box matrix–vector product routine to evaluate the products $\mathbf{Y}_i = \mathbf{A}\boldsymbol{\Omega}_i, i \in \{1, 2, 3, 4\}$, with the following structures. Contained within these products are randomized samples of each admissible block of level 2.

$$\begin{aligned} \mathbf{Y}_1 = \mathbf{A}\boldsymbol{\Omega}_1 &= \begin{bmatrix} * \\ * \\ \mathbf{A}_{6,4}\mathbf{G}_4 \\ \mathbf{A}_{7,4}\mathbf{G}_4 \end{bmatrix}, \qquad \mathbf{Y}_2 = \mathbf{A}\boldsymbol{\Omega}_2 = \begin{bmatrix} * \\ * \\ * \\ \mathbf{A}_{7,5}\mathbf{G}_5 \end{bmatrix} \\ \mathbf{Y}_3 = \mathbf{A}\boldsymbol{\Omega}_3 &= \begin{bmatrix} \mathbf{A}_{4,6}\mathbf{G}_6 \\ * \\ * \\ * \end{bmatrix}, \qquad \mathbf{Y}_4 = \mathbf{A}\boldsymbol{\Omega}_4 = \begin{bmatrix} \mathbf{A}_{4,7}\mathbf{G}_7 \\ \mathbf{A}_{5,7}\mathbf{G}_7 \\ * \\ * \end{bmatrix} \end{aligned} \tag{4.1}$$

We then obtain a basis matrix for the column space of each admissible block by orthonormalizing the relevant block of one of the sample matrices.

$$\begin{aligned} \boldsymbol{\mathcal{U}}_{4,6} &= \mathtt{qr}(\mathbf{Y}_3(I_4, :)) \\ \boldsymbol{\mathcal{U}}_{4,7} &= \mathtt{qr}(\mathbf{Y}_4(I_4, :)) \\ \boldsymbol{\mathcal{U}}_{5,7} &= \mathtt{qr}(\mathbf{Y}_4(I_5, :)) \\ \boldsymbol{\mathcal{U}}_{6,4} &= \mathtt{qr}(\mathbf{Y}_1(I_6, :)) \\ \boldsymbol{\mathcal{U}}_{7,4} &= \mathtt{qr}(\mathbf{Y}_1(I_7, :)) \\ \boldsymbol{\mathcal{U}}_{7,5} &= \mathtt{qr}(\mathbf{Y}_2(I_7, :)) \end{aligned} \tag{4.2}$$

To find a basis matrix for the row space of each admissible block, we follow a similar process using $\mathbf{A}^*$ in place of $\mathbf{A}$. That is, we compute another set of sample matrices $\mathbf{Z}_i = \mathbf{A}^*\boldsymbol{\Psi}_i, i \in \{1, 2, 3, 4\}$, from which we orthonormalize the relevant blocks to obtain row bases $\boldsymbol{\mathcal{V}}_{\alpha,\beta}$ for each admissible pair (α, β).

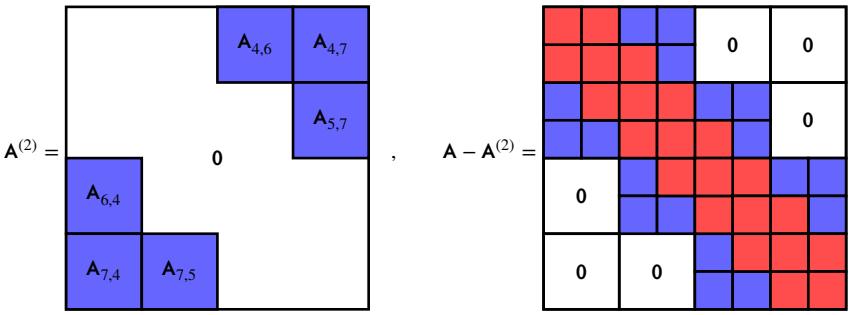
Finally, we solve for the matrices $\mathbf{B}_{\alpha,\beta}$ as follows. Note that the products $\mathbf{A}_{\alpha,\beta}\mathbf{G}_\beta$ have already been obtained from the samples in Eq. (4.1).

$$\mathbf{B}_{\alpha,\beta} = \left(\mathbf{G}_\alpha \boldsymbol{\mathcal{U}}_\alpha\right)^\dagger \mathbf{G}_\alpha \mathbf{A}_{\alpha,\beta} \mathbf{G}_\beta \left(\boldsymbol{\mathcal{V}}_\beta \mathbf{G}_\beta\right)^\dagger \quad \text{for each admissible pair } (\alpha, \beta) \tag{4.3}$$

4.1.2. Compressing levels $3, \dots, L$ 3, …, L

After we have obtained low-rank approximations of the blocks associated with level 2 of the tree, we proceed to level 3. One approach would be to extend the procedure for level 2 by using one test matrix corresponding to each box, for a total of eight test matrices. That approach would grow to be prohibitively expensive for finer levels of the tree as the number of matrix–vector products required would grow proportionately with the number of boxes. Instead, we present a more efficient procedure, which requires a number of matrix–vector products that is bounded across all levels.

Following [23], we define the level-l truncated matrix $\mathbf{A}^{(l)}$ to be the matrix obtained by replacing with zeros every block of $\mathbf{A}$ corresponding to levels finer than level l. Note that a matrix–vector product involving $\mathbf{A}^{(l)}$ can be computed inexpensively using the low-rank approximations already computed when processing levels $2, \dots, l-1$. The structures of the level-2 truncated matrix and the difference $\mathbf{A} - \mathbf{A}^{(2)}$ are shown below.



We will sample the admissible blocks of level 3 by applying $\mathbf{A} - \mathbf{A}^{(2)}$ to a set of test matrices subject to certain conditions on their sparsity structure. For example, to isolate a sample of $\mathbf{A}_{8,10}$ (see Fig. 3), we must avoid unwanted contributions from $\mathbf{A}_{8,8}, \mathbf{A}_{8,9}, \mathbf{A}_{8,11}$, so we multiply $\mathbf{A} - \mathbf{A}^{(2)}$ with a test matrix whose rows indexed by I_8, I_9, I_{11} are all zeros and whose rows indexed by I_{10} are filled with random values drawn from the standard normal distribution. The contents of the other rows are irrelevant for the purpose of sampling $\mathbf{A}_{8,10}$ since they will be multiplied with zeros in $\mathbf{A} - \mathbf{A}^{(2)}$. More generally, to sample some admissible block $\mathbf{A}_{\alpha,\beta}$ of level 3, we require a test matrix $\boldsymbol{\Omega}$ that satisfies the following *sampling constraints*.

$$\begin{aligned} &\boldsymbol{\Omega}(I_\beta, :) = \mathbf{G}_\beta \\ &\boldsymbol{\Omega}(I_\gamma, :) = \mathbf{0} \qquad \text{for all } \gamma \in \mathcal{L}_\alpha^{\text{nei}} \cup \mathcal{L}_\alpha^{\text{int}} \setminus \{\beta\} \end{aligned} \tag{4.4}$$

where $\mathbf{G}_\beta$ is a random matrix of size $|I_\beta|$-by-r. If test matrix $\boldsymbol{\Omega}$ satisfies those sampling constraints, then the rows of the product $\mathbf{A}\boldsymbol{\Omega} - \mathbf{A}^{(2)}\boldsymbol{\Omega}$ indexed by I_α will contain a randomized sample of the column space of $\mathbf{A}_{\alpha,\beta}$.

To sample all of the admissible blocks, we require a set of test matrices $\{\boldsymbol{\Omega}_i\}$ such that for every admissible pair (α, β), there is a test matrix within the set that satisfies the constraints (4.4) associated with that pair. Moreover, we would like that set to be as small as possible to minimize cost.

In order to minimize the number of test matrices, we aim to form a small number of groups of compatible sampling constraints. We say that two sets of sampling constraints are *compatible* if it is possible to form a test matrix $\boldsymbol{\Omega}$ that satisfies both sets of constraints. We then define a *constraint incompatibility graph*, which represents compatibility relationships between pairs of constraint sets. The graph corresponding to the 18 admissible blocks belonging to level 3 is depicted in Fig. 4.

Definition 4.1 (*Constraint Incompatibility Graph*)**.** The constraint incompatibility graph for level l of the tree is the graph in which each vertex corresponds to a distinct constraint set (4.4), and pairs of vertices are connected by an edge if their corresponding constraint sets are incompatible.

We then compute a vertex coloring of the constraint incompatibility graph using the DSatur algorithm (Algorithm 2.3). For a valid coloring of the graph, each subset of vertices sharing the same color represents a mutually compatible collection of sampling constraints. Then for each color, we can define one test matrix that satisfies all of the sampling constraints associated with the vertices of that color. The coloring shown in Fig. 4 yields test matrices with the following structures.

$$\begin{bmatrix} \boldsymbol{\Omega}_1 & \boldsymbol{\Omega}_2 & \boldsymbol{\Omega}_3 & \boldsymbol{\Omega}_4 & \boldsymbol{\Omega}_5 & \boldsymbol{\Omega}_6 \end{bmatrix} = \begin{bmatrix} \mathbf{G}_8 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}_9 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{G}_{10} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G}_{11} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G}_{12} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G}_{13} \\ \mathbf{G}_{14} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}_{15} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \tag{4.5}$$

As in Section 4.1.1, we evaluate the samples $\mathbf{Y}_i = \mathbf{A}\boldsymbol{\Omega}_i - \mathbf{A}^{(2)}\boldsymbol{\Omega}_i$ and orthonormalize the relevant blocks to obtain orthonormal bases $\boldsymbol{\mathcal{U}}_{\alpha,\beta}$ of the column spaces of the admissible blocks. A similar process yields orthonormal bases $\boldsymbol{\mathcal{V}}_{\alpha,\beta}$ of the row spaces. Finally, we solve for $\mathbf{B}_{\alpha,\beta}$ again using (4.3).

4.1.3. Extracting inadmissible blocks of the leaf level

Once we have computed low-rank approximations of the admissible blocks for every level, we finally extract the inadmissible blocks of the leaf level. Since the inadmissible blocks are not necessarily low-rank, they cannot be recovered from a small number of randomized samples. Instead, we use test matrices that will multiply the inadmissible blocks with appropriately sized identity matrices. Also, at this point we already have low-rank approximations of the admissible blocks belonging to level L, so the test matrices only need to avoid contributions from other inadmissible blocks, resulting in fewer constraints than (4.4). To extract inadmissible block $\mathbf{A}_{\alpha,\beta}$ of level L, we require a test matrix $\boldsymbol{\Omega}$ that satisfies the following sampling constraints.

$$\begin{aligned} &\boldsymbol{\Omega}(I_\beta, :) = \mathbf{I} \\ &\boldsymbol{\Omega}(I_\gamma, :) = \mathbf{0} \quad \text{for all } \gamma \in \mathcal{L}_\alpha^{\text{nei}} \setminus \{\beta\} \end{aligned}$$

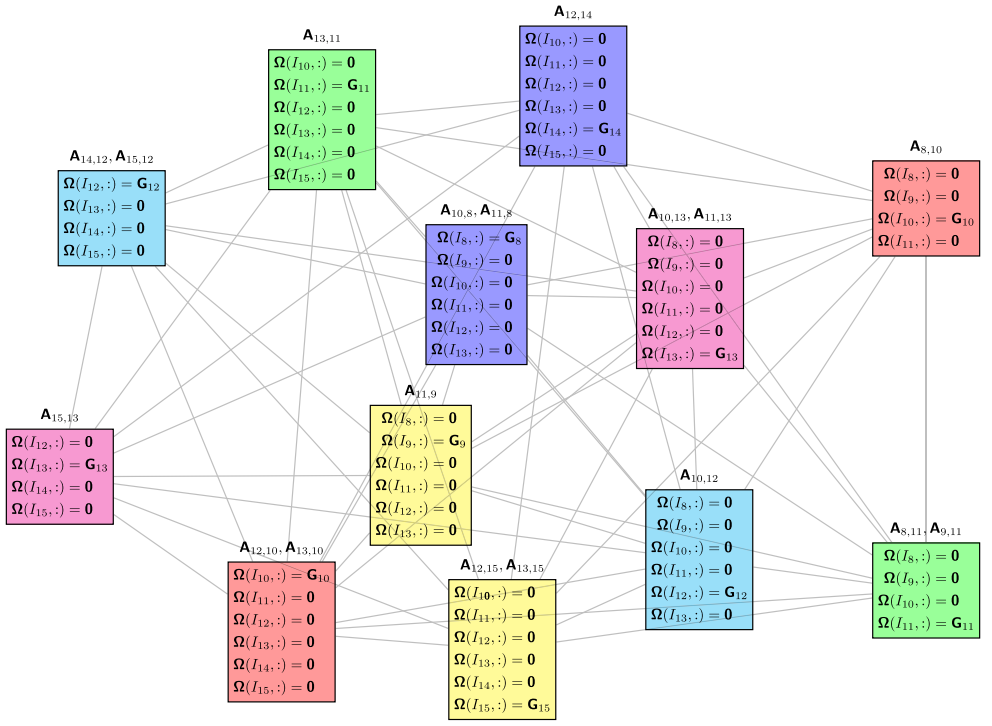


Fig. 4. The constraint incompatibility graph corresponding to the 18 admissible blocks belonging to level 3 of the matrix shown in Fig. 3. Each vertex corresponds to a distinct set of sampling constraints (4.4). Edges connect pairs of vertices that are incompatible. The number of vertices is less than the number of admissible blocks since some admissible blocks share the same set of sampling constraints.

If test matrix $\mathbf{\Omega}$ satisfies these constraints, then the rows of the product $\mathbf{A\Omega} - \mathbf{A}^{(2)}\mathbf{\Omega}$ indexed by I_α will contain $\mathbf{A}_{\alpha,\beta}$. The graph corresponding to the 22 inadmissible blocks belonging to level 3 is depicted in Fig. 5, and its coloring produces test matrices with the following structures.

$$\begin{bmatrix} \mathbf{\Omega}_1 & \mathbf{\Omega}_2 & \mathbf{\Omega}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_8 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_9 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{10} \\ \mathbf{I}_{11} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{12} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{13} \\ \mathbf{I}_{14} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{15} & \mathbf{0} \end{bmatrix}$$

The entire process of compressing an $\mathcal{H}^1$ matrix is summarized in Algorithm 4.1.

4.1.4. *General patterns in the test matrices for $\mathcal{H}^1$ H1 compression*

In this section, we describe sets of test matrices that are sufficient for compressing any $\mathcal{H}^1$ matrix based on points in arbitrary dimensions. Though sufficient, these sets of test matrices are not necessarily the smallest possible, and the proposed method of constructing problem-specific test matrices via graph coloring often yields a smaller set of test matrices, resulting in fewer matrix–vector products. The test matrices described in this section establish an upper bound on the number of matrix–vector products that improves on the number given in [20], and they also imply a bound on the chromatic number of the graph, which appears in the estimate of asymptotic complexity.

A general set of test matrices for sampling admissible blocks. The structures of the test matrices derived in Sections 4.1.2 and 4.1.3 exhibit patterns that generalize to finer and higher-dimensional grids. The sampling constraints Eq. (4.4) for admissible block $\mathbf{A}_{\alpha,\beta}$ specify that the rows of the test matrix corresponding to β must be filled with random values, and the rows of the test matrix corresponding to the other boxes in $\mathcal{L}_\alpha^{\text{nei}} \cup \mathcal{L}_\alpha^{\text{int}}$ must be filled with zeros. For a 1-dimensional problem, the indices corresponding to $\mathcal{L}_\alpha^{\text{nei}} \cup \mathcal{L}_\alpha^{\text{int}}$ form 6 contiguous blocks of the test matrix. Accordingly, each of the test matrices defined in Eq. (4.1) has every sixth block filled with random values and the other blocks filled with zeros, a pattern that generalizes to an arbitrarily fine grid in one dimension.

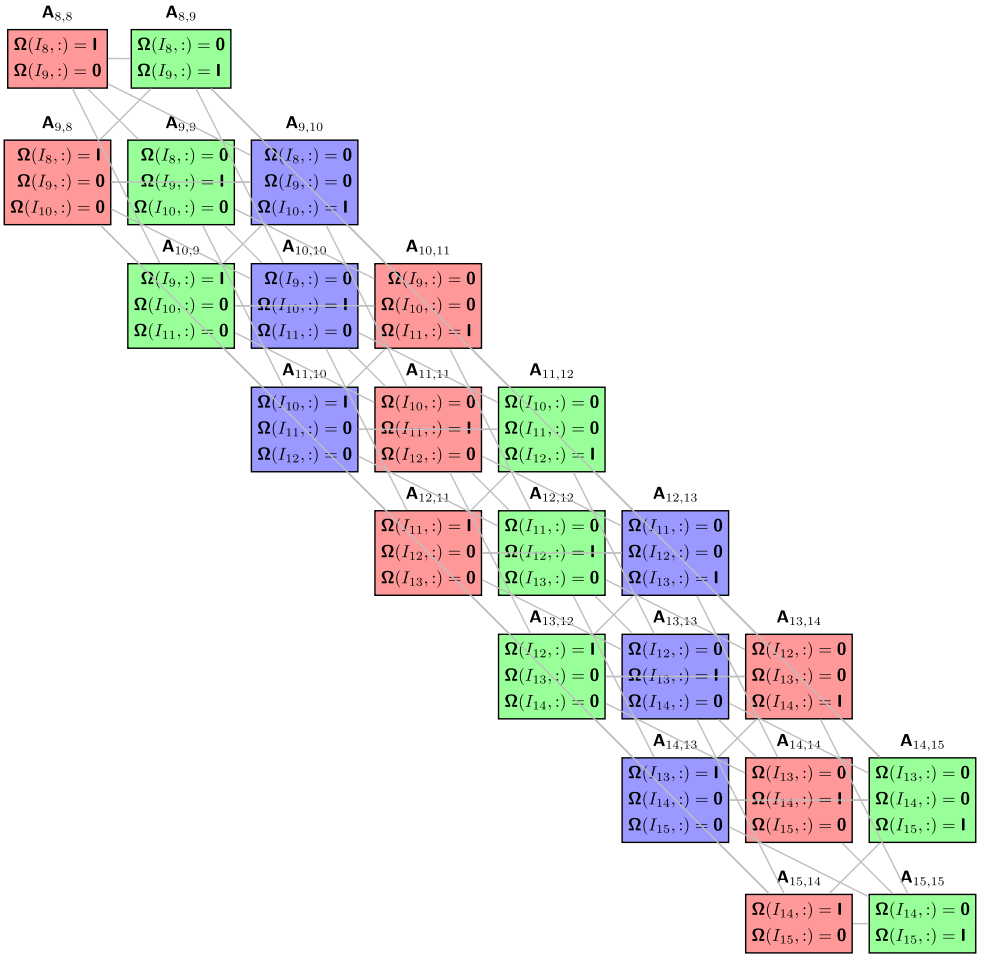


Fig. 5. Incompatibility graph for inadmissible blocks belonging to level L.

To describe a general set of test matrices for a problem in 2 dimensions, we partition the domain into tiles, each of which covers 6×6 boxes. We define 36 test matrices, each activating a set of boxes that share the same position within their respective tiles. The set of boxes activated by one of the test matrices is shown in Fig. 6. The sampling constraints in Eq. (4.4) apply to $\mathcal{L}_\alpha^{\text{nei}} \cup \mathcal{L}_\alpha^{\text{int}}$, which form a square of 6×6 boxes, and they specify that exactly one of those boxes must be activated. Therefore, those constraints will be satisfied by one of the 36 test matrices. By a similar argument, this pattern generalizes to higher-dimensional problems, requiring at most 6^d test matrices to sample one level of admissible blocks for a problem in d dimensions. Furthermore, since these test matrices satisfy Eq. (4.4), they must correspond to a valid coloring of the graph described in Section 4.1.2, bounding the chromatic number of the graph by

$$\chi_{\text{nonunif}} \leq 6^d.$$

A general set of test matrices for extracting inadmissible blocks. A similar argument proves that extracting inadmissible blocks of the leaf level can be accomplished using 3^d test matrices. The set of boxes activated by one of the test matrices for a problem in two dimensions is shown in Fig. 6. Therefore, the chromatic number of the graph described in Section 4.1.3 is bounded by

$$\chi_{\text{leaf}} \leq 3^d.$$

4.1.5. Asymptotic complexity

Let $L \sim \log N$ denote the depth of the tree, T_{mult} denote the time to apply $\mathbf{A}$ or $\mathbf{A}^*$ to a vector, and T_{flop} denote the time to execute one floating point operation. There are 2^{dl} boxes belonging to level l of the tree, and the interaction list of each box consists of up to $6^d - 3^d$ other boxes, so there are approximately $(6^d - 3^d)2^{dl}$ admissible blocks associated with level l. The cost of applying the low-rank approximation of an admissible block associated with level l to a vector is $\sim kN/2^{dl}$ operations. Therefore, the cost of

Algorithm 4.1 Randomized compression of an $\mathcal{H}^1$ matrix

for level $l \in [2, ..., L]$ **do**

Compute randomized samples of $\mathbf{A} - \mathbf{A}^{(l)}$

Construct structured random test matrices $\{\mathbf{\Omega}_i\}$ of size $N \times (k+p)$ as in Section 4.1.2 or Section 4.1.4

for all $\mathbf{\Omega}_i$ **do**

Multiply $\mathbf{Y}_i = \mathbf{A}\mathbf{\Omega}_i - \mathbf{A}^{(l)}\mathbf{\Omega}_i$

Compute orthonormal basis matrices $\mathcal{U}_{\alpha,\beta}$

for all interacting pairs α, β in level l **do**

Identify $\mathbf{Y}_i$ that contains a sample of $\mathbf{A}_{\alpha,\beta}$ in $\mathbf{Y}_i(I_\alpha, :)$

Orthonormalize $\mathcal{U}_{\alpha,\beta} = \mathtt{qr}(\mathbf{Y}_i(I_\alpha, :), k)$

Compute randomized samples of $\mathbf{A}^* - \mathbf{A}^{(l)*}$

Construct structured random test matrices $\{\mathbf{\Psi}_i\}$ of size $N \times (k+p)$ as in Section 4.1.2 or Section 4.1.4 (if the rank structure of $\mathbf{A}$ is symmetric, then the test matrices $\{\mathbf{\Omega}_i\}$ can be reused)

for all $\mathbf{\Psi}_i$ **do**

Multiply $\mathbf{Z}_i = \mathbf{A}^*\mathbf{\Psi}_i - \mathbf{A}^{(l)*}\mathbf{\Psi}_i$

Compute orthonormal basis matrices $\mathcal{V}_{\alpha,\beta}$

for all interacting pairs α, β in level l **do**

Identify $\mathbf{Z}_i$ that contains a sample of $\mathbf{A}^*_{\alpha,\beta}$ in $\mathbf{Z}_i(I_\beta, :)$

Orthonormalize $\mathcal{V}_{\alpha,\beta} = \mathtt{qr}(\mathbf{Z}_i(I_\beta, :), k)$

Solve for $\mathbf{B}$

for all interacting pairs α, β in level l **do**

$\mathbf{B}_{\alpha,\beta} = \left(\mathbf{G}_\alpha \mathcal{U}_\alpha\right)^\dagger \mathbf{G}_\alpha \mathbf{A}_{\alpha,\beta} \mathbf{G}_\beta \left(\mathcal{V}_\beta \mathbf{G}_\beta\right)^\dagger$

Extract the inadmissible blocks of level L

Construct structured random test matrices $\{\mathbf{\Omega}_i\}$ of size $N \times m$ as in Section 4.1.3 or Section 4.1.4

for all $\mathbf{\Omega}_i$ **do**

Multiply $\mathbf{Y}_i = \mathbf{A}\mathbf{\Omega}_i - \mathbf{A}^{(L)}\mathbf{\Omega}_i$

for all neighbor pairs α, β in level L **do**

Identify $\mathbf{Y}_i$ that contains $\mathbf{A}_{\alpha,\beta}$

Extract the block $\mathbf{A}_{\alpha,\beta}$ from $\mathbf{Y}_i(I_\alpha, :)$

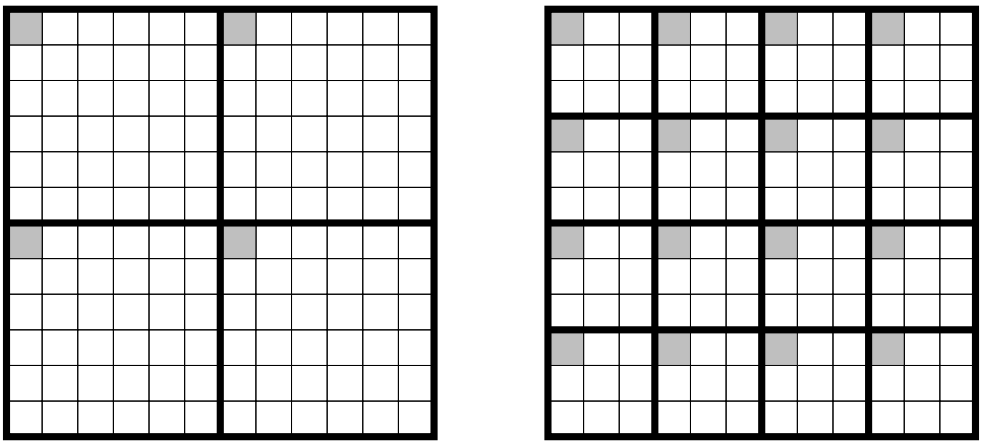


Fig. 6. Patterns representing a test matrix for sampling admissible blocks (left) and a test matrix for sampling inadmissible blocks (right) for a problem in 2 dimensions. Blocks of the test matrices corresponding to gray boxes are filled with random values, and those corresponding to white boxes are filled with zeros. The other test matrices are obtained by shifting these pattern horizontally and vertically.

applying the level-l truncated matrix $\mathbf{A}^{(l)}$ (or its transpose) to a vector is

$$T_{A^{(l)}} \sim T_{\text{flop}} \times \sum_{j=0}^{l} (6^d - 3^d) 2^{dj} k \frac{N}{2^{dj}} \sim T_{\text{flop}} \times (6^d - 3^d) l k N.$$

The cost of compressing the admissible blocks associated with level l of the tree is

$$T_l \sim \left(T_{\text{mult}} + T_{A^{(l)}}\right) \times 2\chi_{\text{nonunif}} k + T_{\text{flop}} \times (6^d - 3^d) 2^{dl} k^2 \frac{N}{2^{dl}},$$

since we require $\sim \chi_{\text{nonunif}} k$ applications of $\mathbf{A} - \mathbf{A}^{(l)}$ and its transpose, where χ_{nonunif} denotes the chromatic number of the graph described in Section 4.1.2, and we require an additional $\sim k^2 N / 2^{dl}$ operations for each admissible block to compute low-rank approximations.

Finally, summing T_l over each level gives the total time to construct an $\mathcal{H}^1$ representation as follows. The cost of extracting inadmissible blocks associated with the leaf level of the tree is omitted as it only contributes a lower order term to the overall cost. We also omit the costs associated with the graph coloring problem since it is only worthwhile for problems that exhibit low-dimensional structure, in which case the costs would be much lower than the worst-case analysis would suggest, and we observe in practice that the cost of graph coloring represents a small portion of the total cost of compression. For problems without low-dimensional structure, one may skip the graph coloring step and instead use the general set of test matrices described in Section 4.1.4.

$$T_{\text{compress}} \sim T_{\text{mult}} \times 2\chi_{\text{nonunif}} k \log N + T_{\text{flop}} \times (6^d - 3^d) 2 \chi_{\text{nonunif}} k^2 N (\log N)^2$$

4.2. Uniform $\mathcal{H}^1$ H1 matrix compression

A uniform $\mathcal{H}^1$ approximation requires for each box τ a column basis matrix $\mathcal{U}_\tau$ and a row basis matrix $\mathcal{V}_\tau$ such that

$$\mathbf{A}_{\alpha,\beta} \approx \mathcal{U}_\alpha \mathcal{U}_\alpha^* \mathbf{A}_{\alpha,\beta} \mathcal{V}_\beta \mathcal{V}_\beta^*$$

for all admissible pairs (α, β). In other words, $\mathcal{U}_\alpha$ must span the column space of $\mathbf{A}\left(I_\alpha, \cup_{\beta \in \mathcal{L}_\alpha^{\text{int}}} I_\beta\right)$, the submatrix of interactions between α and the boxes in its interaction list. Similarly, $\mathcal{V}_\beta$ must span the row space of $\mathbf{A}\left(\cup_{\alpha \in \mathcal{L}_\beta^{\text{int}}} I_\alpha, I_\beta\right)$. The algorithms for compressing $\mathcal{H}^1$ and uniform $\mathcal{H}^1$ matrices have a two important differences. For a uniform $\mathcal{H}^1$ matrix, we use Algorithm 2.2, rather than Algorithm 2.1 to compute low-rank approximations. Also, instead of sampling the column space of each admissible block $\mathbf{A}_{\alpha,\beta}$ separately, we will sample the interactions between each box and all of the boxes in its interaction list together.

We return to the task of compressing the admissible blocks associated with level 3 of the tree as described in Section 4.1.2. As before, we will sample $\mathbf{A} - \mathbf{A}^{(2)}$ with a set of test matrices. To sample those interactions for some box α, we require a test matrix $\mathbf{\Omega}$ that satisfies the following sampling constraints.

$$\begin{aligned} \mathbf{\Omega}(I_\beta, :) &= \mathbf{G}_\beta \quad \text{for all } \beta \in \mathcal{L}_\alpha^{\text{int}} \\ \mathbf{\Omega}(I_\gamma, :) &= \mathbf{0} \quad \text{for all } \gamma \in \mathcal{L}_\alpha^{\text{nei}} \end{aligned} \tag{4.6}$$

If test matrix $\mathbf{\Omega}$ satisfies the above sampling constraints, then the rows of the product $\mathbf{A\Omega} - \mathbf{A}^{(2)}\mathbf{\Omega}$ indexed by I_α will contain a randomized sample of the column space of $\mathbf{A}\left(I_\alpha, \cup_{\beta \in \mathcal{L}_\alpha^{\text{int}}} I_\beta\right)$, the submatrix of interactions between α and the boxes in its interaction list.

We have one set of sampling constraints for each box α, and we use them to form the constraint incompatibility graph (Definition 4.1) shown in Fig. 7. As in Section 4.1.2, we compute a vertex coloring of the graph. The coloring of the graph in Fig. 7 specifies test matrices with the following structures.

$$\begin{bmatrix} \mathbf{\Omega}_1 & \mathbf{\Omega}_2 & \mathbf{\Omega}_3 & \mathbf{\Omega}_4 & \mathbf{\Omega}_5 \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{G}_8 & \mathbf{G}_8 & * \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G}_9 & * \\ \mathbf{G}_{10} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G}_{10} \\ \mathbf{G}_{11} & \mathbf{G}_{11} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}_{12} & \mathbf{G}_{12} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{G}_{13} & \mathbf{G}_{13} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & * & \mathbf{G}_{14} \\ \mathbf{G}_{15} & \mathbf{0} & \mathbf{0} & * & \mathbf{G}_{15} \end{bmatrix}$$

We evaluate the samples $\mathbf{Y}_i = \mathbf{A\Omega}_i - \mathbf{A}^{(2)}\mathbf{\Omega}_i$ for each test matrix $\mathbf{\Omega}_i$, and orthonormalize the relevant blocks of the sample matrices $\mathbf{Y}_i$ to obtain orthonormal column basis matrix $\mathcal{U}_\alpha$ for each box α. That is, if $\mathbf{\Omega}_i$ is the test matrix satisfying the sampling constraints of box α, then

$$\mathbf{Y}_i(I_\alpha, :) = \sum_{\beta \in \mathcal{L}_\alpha^{\text{int}}} \mathbf{A}_{\alpha,\beta} \mathbf{G}_\beta,$$

so we compute

$$\mathcal{U}_\alpha = \texttt{qr}\left(\mathbf{Y}_i(I_\alpha, :)\right).$$

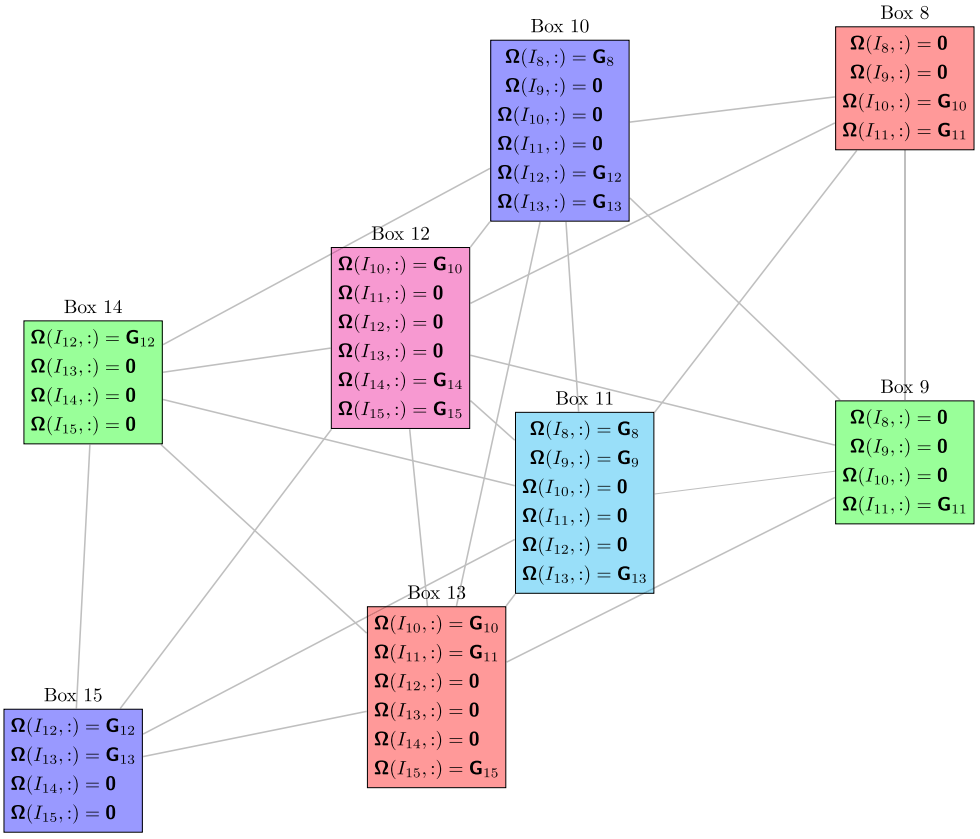


Fig. 7. The constraint incompatibility graph corresponding to the 8 boxes on level 3 of the matrix shown in Fig. 3 along with the constraints (4.6) associated with sampling a uniform $\mathcal{H}^1$ matrix.

For the second stage of compression using Algorithm 2.2, we must compute $\mathbf{A}_{\alpha,\beta}^* \mathcal{U}_\alpha$ for each admissible block $\mathbf{A}_{\alpha,\beta}$. To do so, we use the procedure for sampling an $\mathcal{H}^1$ matrix with non-uniform basis matrices (Section 4.1.2), but rather than filling blocks of the test matrices with random values, we fill them with appropriately chosen uniform basis matrices. With that modification, we have the following sampling constraints for each admissible pair (α, β).

$$\mathbf{\Psi}(I_\beta, :) = \mathcal{U}_\beta$$
$$\mathbf{\Psi}(I_\gamma, :) = \mathbf{0} \qquad \text{for all } \gamma \in \mathcal{L}_\alpha^{\text{nei}} \cup \mathcal{L}_\alpha^{\text{int}} \setminus \{\beta\}$$

We construct a suitable set of test matrices via graph coloring and evaluate the products $\mathbf{Z}_i = \left(\mathbf{A} - \mathbf{A}^{(2)}\right)^* \mathbf{\Psi}_i$ for each test matrix $\mathbf{\Psi}_i$. Then for each admissible pair (α, β), there is some $\mathbf{Z}_i$ such that $\mathbf{Z}_i(I_\beta, :) = \mathbf{A}_{\alpha,\beta}^* \mathcal{U}_\alpha$. With those results, we compute the uniform basis matrices $\mathcal{V}_\alpha$ and the matrices $\mathbf{B}_{\alpha,\beta}$ as follows.

$$\mathcal{V}_\beta = \texttt{qr}\left(\sum_{\alpha \in \mathcal{L}_\beta^{\text{int}}} \mathbf{Z}_i(I_\beta, :) \right) = \texttt{qr}\left(\sum_{\alpha \in \mathcal{L}_\beta^{\text{int}}} \mathbf{A}_{\alpha,\beta}^* \mathcal{U}_\alpha \right)$$
$$\mathbf{B}_{\alpha,\beta} = \mathbf{Z}_i(I_\beta, :)^* \mathcal{V}_\beta = (\mathbf{A}_{\alpha,\beta}^* \mathcal{U}_\alpha)^* \mathcal{V}_\beta$$

Note that it would have been possible to construct $\mathcal{V}_\beta$ by taking combined samples of the interactions between β and all of the boxes in its interaction list, similarly to how we obtain $\mathcal{U}_\beta$. However, then we would not have $\mathbf{A}_{\alpha,\beta}^* \mathcal{U}_\alpha$, which is later required when computing $\mathbf{B}_{\alpha,\beta}$. The advantage of sampling $\mathbf{A}^*$ the way we do is that we obtain what is needed to compute both $\mathcal{V}_\beta$ and $\mathbf{B}_{\alpha,\beta}$.

Remark 4.2. An alternative approach for uniform $\mathcal{H}^1$ compression [20] is obtained by adding a uniformization step to the $\mathcal{H}^1$ compression algorithm. With that approach, one first computes low-rank approximations $\mathbf{A}_{\alpha,\beta} = \mathcal{U}_{\alpha,\beta} \mathbf{B}_{\alpha,\beta} \mathcal{V}_{\alpha,\beta}^*$ with non-uniform basis matrices. Those low-rank approximations are recompressed to obtain uniform basis matrices, followed by a change of basis.

$$\mathbf{A}_{\alpha,\beta} \approx \mathcal{U}_\alpha \left(\mathcal{U}_\alpha^* \mathcal{U}_{\alpha,\beta} \mathbf{B}_{\alpha,\beta} \mathcal{V}_{\alpha,\beta}^* \mathcal{V}_\beta \right) \mathcal{V}_\beta^*$$

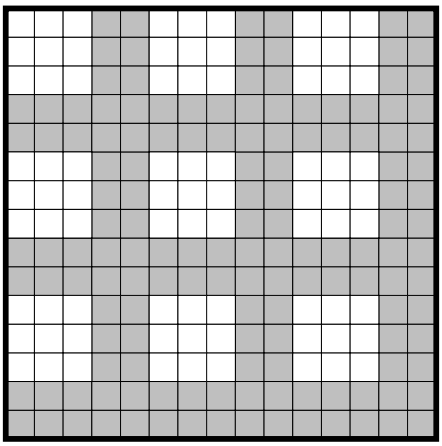


Fig. 8. A pattern representing one test matrix for the first stage of uniform $\mathcal{H}^1$ sampling for a problem in 2 dimensions. Blocks of the test matrix corresponding to gray boxes are filled with random values, and those corresponding to white boxes are filled with zeros. The other test matrices are obtained by shifting this pattern horizontally and vertically.

One disadvantage of that approach is that $\mathcal{U}_\alpha$ and $\mathcal{V}_\alpha$ are based on samples of the factorizations $\mathcal{U}_{\alpha,\beta}\mathbf{B}_{\alpha,\beta}\mathcal{V}^*_{\alpha,\beta}$, which may only be approximate, rather than direct samples of the blocks $\mathbf{A}_{\alpha,\beta}$ themselves, potentially resulting in lower accuracy. Another difference is that the procedure for sampling a uniform $\mathcal{H}^1$ matrix presented in this section requires fewer matrix–vector products than the procedure for sampling an $\mathcal{H}^1$ matrix, as described in Sections 4.1.4 and 4.2.1.

4.2.1. General patterns in the test matrices for uniform $\mathcal{H}^1$ H1 compression

As in Section 4.1.4, we describe a general set of test matrices that is applicable to finer and higher-dimensional grids. To sample the interactions of box α for a problem in 2 dimensions, the blocks of the test matrix corresponding to the boxes in $\mathcal{L}^{\text{nei}}_\alpha$ must be filled with zeros, and the blocks corresponding to the boxes in $\mathcal{L}^{\text{int}}_\alpha$ must be filled with random values Eq. (4.6). The pattern of activated boxes for one test matrix is shown in Fig. 8. The complete set of 25 test matrices is obtained by shifting the pattern horizontally and vertically. The pattern generalizes to higher-dimensional problems, requiring at most 5^d test matrices to carry out the first stage of uniform $\mathcal{H}^1$ sampling for a problem in d dimensions. Therefore, we establish the upper bound

$$\chi_{\text{unif}} \leq 5^d.$$

4.3. $\mathcal{H}^2$ H2 matrix compression

To obtain an algorithm for computing an $\mathcal{H}^2$ representation, we apply three modifications to the algorithm for computing a uniform $\mathcal{H}^1$ representation. First, we take singular value decompositions of samples, rather than orthonormalizing via QR factorizations. Second, we enrich the basis matrices of each box so that they span the relevant parts of the basis matrices of its parent. For box α with parent τ, we define $I_{\tau,\alpha}$ to be the positions of the indices of I_α within I_τ so that $I_\tau(I_{\tau,\alpha}) = I_\alpha$. When computing the basis matrix $\mathcal{U}_\alpha$, we augment $\mathbf{Y}_i(I_\alpha, :)$, the sample of $\mathbf{A}\left(I_\alpha, \cup_{\beta\in\mathcal{L}^{\text{int}}_\alpha} I_\beta\right)$, with $\mathcal{U}_\tau(I_{\tau,\alpha}, :)\mathbf{\Sigma}^{\text{in}}_\tau\mathbf{G}_k$, a sample of $\mathbf{A}\left(I_\alpha, \left(\cup_{\beta\in\mathcal{L}^{\text{nei}}_\tau} I_\beta\right)^c\right)$, where $\mathbf{G}_k$ is a k-by-k Gaussian random matrix.

$$\mathbf{Y}_i(I_\alpha, :) = \sum_{\beta\in\mathcal{L}^{\text{int}}_\alpha} \mathbf{A}_{\alpha,\beta}\mathbf{G}_\beta$$

$$[\mathcal{U}_\alpha, \mathbf{\Sigma}^{\text{in}}_\alpha, \sim] = \texttt{svd}\left(\mathbf{Y}_i(I_\alpha, :) + \mathcal{U}_\tau(I_{\tau,\alpha}, :)\mathbf{\Sigma}^{\text{in}}_\tau\mathbf{G}_k\right)$$

We use a similarly augmented sample when computing $\mathcal{V}_\alpha$.

$$[\mathcal{V}_\alpha, \mathbf{\Sigma}^{\text{out}}_\alpha, \sim] = \texttt{svd}\left(\left(\sum_{\beta\in\mathcal{L}^{\text{int}}_\alpha} \mathbf{A}^*_{\beta,\alpha}\mathcal{U}_\beta\right) + \mathcal{V}_\tau(I_{\tau,\alpha}, :)\mathbf{\Sigma}^{\text{out}}_\tau\mathbf{G}_k\right)$$

For some boxes, there may be no contribution from the parent with which to augment the sample. For example, if α is a box belonging to level 2, then $\left(\cup_{\beta\in\mathcal{L}^{\text{nei}}_\tau} I_\beta\right)^c$ is the empty set. In such cases, we compute the SVD without augmenting. Third, after we have computed long basis matrices of the children of τ, we compute the short basis matrix $\mathbf{U}_\tau$ by solving (3.1), and then we can discard $\mathcal{U}_\tau$. Similarly, we compute $\mathbf{V}_\tau$ and discard $\mathcal{V}_\tau$. The processes of compressing uniform $\mathcal{H}^1$ and $\mathcal{H}^2$ matrices are summarized in Algorithm 4.2. The algorithm for applying a level-truncated approximation of an $\mathcal{H}^2$ matrix to a vector is given in Algorithm 4.3.

Algorithm 4.2 Randomized compression of a uniform $\mathcal{H}^1$ or $\mathcal{H}^2$ matrix

for level $l \in [2, ..., L]$ **do**

Compute randomized samples of $\mathbf{A} - \mathbf{A}^{(l)}$

Construct structured random test matrices $\{\mathbf{\Omega}_i\}$ of size $N \times (k+p)$ as in Section 4.2 or Section 4.2.1

for all $\mathbf{\Omega}_i$ **do**

Multiply $\mathbf{Y}_i = \mathbf{A}\mathbf{\Omega}_i - \mathbf{A}^{(l)}\mathbf{\Omega}_i$

Compute uniform orthonormal basis matrices $\mathcal{U}_\alpha$

for all boxes α in level l **do**

Identify $\mathbf{Y}_i$ that contains a sample of $\mathbf{A}(I_\alpha, \cup_{\beta \in \mathcal{L}_\alpha^{\text{int}}} I_\beta)$ in $\mathbf{Y}_i(I_\alpha, :)$

if $\mathbf{A}$ has uniform $\mathcal{H}^1$ structure **then**

Orthonormalize the sample: $\mathcal{U}_\alpha = \texttt{qr}(\mathbf{Y}_i(I_\alpha, :), k)$

else if $\mathbf{A}$ has $\mathcal{H}^2$ structure **then**

With τ as the parent of α, compute an SVD of the augmented sample:
$[\mathcal{U}_\alpha, \mathbf{\Sigma}_\alpha^{\text{in}}, \sim] = \texttt{svd}(\mathbf{Y}_i(I_\alpha, :) + \mathcal{U}_\tau(I_{\tau,\alpha}, :)\mathbf{\Sigma}_\tau^{\text{in}}\mathbf{G}_k, k)$

Compute randomized samples of $\mathbf{A}^* - \mathbf{A}^{(l)*}$

Construct structured random test matrices $\{\mathbf{\Psi}_i\}$ of size $N \times (k+p)$ as in Section 4.1.2 or Section 4.1.4 with the modification in Section 4.2

for all $\mathbf{\Psi}_i$ **do**

Multiply $\mathbf{Z}_i = \mathbf{A}^*\mathbf{\Psi}_i - \mathbf{A}^{(l)*}\mathbf{\Psi}_i$

Compute uniform orthonormal basis matrices $\mathcal{V}_\beta$

for all boxes β in level l **do**

Identify $\mathbf{Z}_i$ that contains $\mathbf{A}_{\alpha,\beta}^*\mathcal{U}_\alpha$ in $\mathbf{Z}_i(I_\beta, :)$ for $\alpha \in \mathcal{L}_\beta^{\text{int}}$

if $\mathbf{A}$ has uniform $\mathcal{H}^1$ structure **then**

Sum samples and orthonormalize: $\mathcal{V}_\beta = \texttt{qr}(\sum_{\alpha \in \mathcal{L}_\beta^{\text{int}}} \mathbf{Z}_i(I_\beta, :), k)$

else if $\mathbf{A}$ has $\mathcal{H}^2$ structure **then**

With τ as the parent of β, sum samples, augment, and compute an SVD:
$[\mathcal{V}_\beta, \mathbf{\Sigma}_\beta^{\text{out}}, \sim] = \texttt{svd}((\sum_{\alpha \in \mathcal{L}_\beta^{\text{int}}} \mathbf{Z}_i(I_\beta, :)) + \mathcal{V}_\tau(I_{\tau,\beta}, :)\mathbf{\Sigma}_\tau^{\text{out}}\mathbf{G}_k, k)$

Compute $\mathbf{B}_{\alpha,\beta}$

for all interacting pairs α, β in level l **do**

Identify $\mathbf{Z}_i$ that contains $\mathbf{A}_{\alpha,\beta}^*\mathcal{U}_\alpha$ in $\mathbf{Z}_i(I_\beta, :)$ for $\alpha \in \mathcal{L}_\beta^{\text{int}}$

$\mathbf{B}_{\alpha,\beta} = \mathbf{Z}_i^*(I_\beta, :)\mathcal{V}_\beta$

Nest the basis matrices

if $\mathbf{A}$ has $\mathcal{H}^2$ structure **then**

for all boxes α in level l **do**

Compute $\mathbf{U}_\alpha, \mathbf{V}_\alpha$ using Equation (3.1)

Discard $\mathcal{U}_\alpha, \mathcal{V}_\alpha$

Extract the inadmissible blocks of level L in the same way as in Algorithm 4.1

4.3.1. Asymptotic complexity

The asymptotic complexity of the $\mathcal{H}^2$ algorithm is similar to that of the $\mathcal{H}^1$ algorithm. Since we are now using uniform basis matrices, the cost of applying $T_{A^{(l)}}$ is lower.

$$\begin{aligned} T_{A^{(l)}} &\sim T_{\text{flop}} \times \left(2^{dl} k \frac{N}{2^{dl}} + \sum_{j=0}^{l} (6^d - 3^d) k^2 2^{dj} \right) \\ &\sim T_{\text{flop}} \times \left(kN + (6^d - 3^d) k^2 2^{dl} \right) \end{aligned}$$

The number of matrix–vector products to carry out sampling for uniform basis matrices is also lower.

$$T_l \sim \left(T_{\text{mult}} + T_{A^{(l)}} \right) \times (\chi_{\text{unif}} + \chi_{\text{nonunif}}) k + T_{\text{flop}} \times (6^d - 3^d) 2^{dl} k^2 \frac{N}{2^{dl}},$$

Algorithm 4.3 Applying an $\mathcal{H}^2$ level-truncated approximation $\mathbf{A}^{(l)}$ to vector $\mathbf{q}$

Build outgoing expansions in level l.
for all boxes τ in level l **do**
$\hat{\mathbf{q}}_\tau = \mathcal{V}_\tau^* \mathbf{q}(I_\tau)$

Build outgoing expansions for levels coarser than l (upward pass).
for all levels $i \in [l-1, ...2]$ **do**
for all boxes τ in level i **do**
for all children α of τ **do**
$\hat{\mathbf{q}}_\tau(I_\alpha, :) = \mathbf{V}_\tau^*(I_\alpha, :)\hat{\mathbf{q}}_\alpha$

Build incoming expansions for boxes in level 2.
for all boxes α in level 2 **do**
$\hat{\mathbf{u}}_\alpha = \sum_{\beta \in \mathcal{L}_\alpha^{\text{int}}} \mathbf{B}_{\alpha,\beta}\hat{\mathbf{q}}_\beta$

Build incoming expansions for levels finer than 2 (downward pass).
for all levels $i \in [3, ..., l-1]$ **do**
for all boxes α in level i **do**
Let τ be the parent of α
$\hat{\mathbf{u}}_\alpha = \sum_{\beta \in \mathcal{L}_\alpha^{\text{int}}} \mathbf{B}_{\alpha,\beta}\hat{\mathbf{q}}_\beta + \mathbf{U}_\tau(I_\alpha, :)\hat{\mathbf{u}}_\tau$

Build incoming expansions for level l.
for all boxes α in level l **do**
$\mathbf{u}(I_\alpha) = \mathcal{U}_\tau \hat{\mathbf{u}}_\tau + \sum_{\beta \in \mathcal{L}_\alpha^{\text{nei}}} \mathbf{A}(I_\alpha, I_\beta)\mathbf{q}(I_\beta)$

Summing T_l over each level, we find that the number of floating point operations is lower than that of Section 4.1.5 by a factor of $\mathcal{O}(\log N)$. As in Section 4.1.5, we omit the costs associated with graph coloring.

$$\begin{aligned} T_{\text{compress}} &\sim T_{\text{mult}} \times \left(\chi_{\text{unif}} + \chi_{\text{nonunif}}\right) k \log N \\ &+ T_{\text{flop}} \times k^2 N \left(\left(\chi_{\text{unif}} + \chi_{\text{nonunif}} + 6^d - 3^d\right) \log N \right. \\ &\left. + (6^d - 3^d)(\chi_{\text{unif}} + \chi_{\text{nonunif}})\right) \end{aligned}$$

5. Numerical experiments

In this section, we present a selection of numerical results. The experiment in Section 5.1 demonstrates the ability of the graph coloring approach to exploit low-dimensional structure. In Sections 5.2–5.5, we report the following quantities for a number of test problems and rank structure formats: (1) the time to compress the operator, (2) the time to apply the compressed representation to a vector, (3) the relative accuracy of the compressed representation, and (4) the storage requirements of the compressed representation measured as the number of floating point values per degree of freedom. For compression time, we report both the total time taken for compression as well as the "net time", which does not include the time spent by the black-box multiplication routine. The algorithms for compressing matrices and applying compressed representations are written in Python, and the black-box multiplication routines are written in MATLAB. The experiments were carried out on a workstation with two Intel Xeon Gold 6254 processors with 18 cores each and 754 GB of memory.

The matrices are compressed with the $\mathcal{H}^1$, uniform $\mathcal{H}^1$, and $\mathcal{H}^2$ formats. For the uniform $\mathcal{H}^1$ format, we take two approaches: the "$\mathcal{H}^1$ + unif." approach uses the $\mathcal{H}^1$ sampling procedure followed by a uniformization step (as in [20]), and the "unif. $\mathcal{H}^1$" approach uses the technique described in Section 4.2, which uses a procedure that samples entire interaction lists and thus avoids the uniformization step.

We measure the accuracy of the compressed matrices using the relative error

$$E = \frac{\|\mathbf{A}_{\text{computed}} - \mathbf{A}\|}{\|\mathbf{A}\|}$$

computed via 20 iterations of the power method. We also report the maximum leaf node size m and the number r of random vectors per test matrix, which are inputs to the compression algorithm.

5.1. Exploiting low-dimensional structure

A major advantage of the graph coloring approach is that it tailors the test matrices to the problem at hand, exploiting low-dimensional structure to minimize the number of matrix–vector products. In Sections 4.1–4.3, we establish upper bounds on the

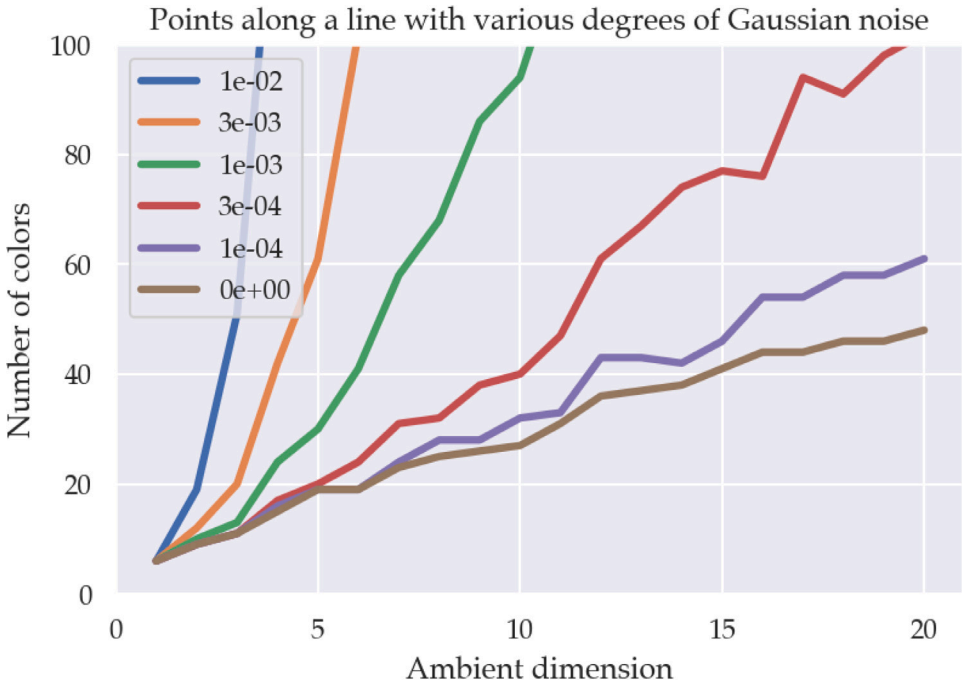


Fig. 9. Number of colors for the incompatibility graphs that arise from sampling one level of admissible blocks of an $\mathcal{H}^1$ matrix based on a uniform grid along a randomly oriented line through $[0,1]^d$ with added perturbation over a range of dimensions d.

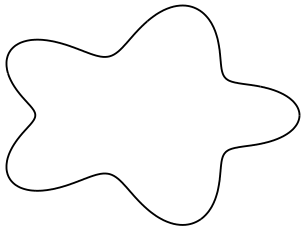


Fig. 10. Contour Γ on which the BIE (5.1) is defined.

chromatic numbers of the graphs in terms of the dimension d of the computational domain. However, if the geometry of the points exhibits lower-dimensional structure (e.g., a discretization of a surface in three-dimensional space, a machine learning dataset consisting of observations belonging to a high-dimensional feature space), the chromatic numbers may be much lower.

To demonstrate this effect, we color the graphs that arise from sampling one level of admissible blocks of an $\mathcal{H}^1$ matrix based on a uniform grid along a randomly oriented line through $[0,1]^d$ over a range of dimensions d. We perturb the data by adding Gaussian noise to the coordinates of the points in order to simulate geometries that are not perfectly one-dimensional. With zero noise, the points lie exactly on a line. As more and more noise is added, the effective dimensionality of the data increases from one to the full ambient dimension. The results reported in Fig. 9 demonstrate that the number of colors grows modestly in the presence of low-dimensional structure and very rapidly in the absence of low-dimensional structure.

5.2. Boundary integral equation

We consider a matrix arising from the discretization of the Boundary Integral Equation (BIE)

$$\frac{1}{2}q(x) + \int_{\Gamma} \frac{(x-y)\cdot n(y)}{4\pi|x-y|^2}\, q(y)\, ds(y) = f(x), \qquad x \in \Gamma, \tag{5.1}$$

where Γ is the simple closed contour in the plane shown in Fig. 10, and where $n(y)$ is the outwards pointing unit normal of Γ at y. The BIE (5.1) is a standard integral equation formulation of the Laplace equation with boundary condition f on the domain interior to Γ. The BIE (5.1) is discretized using the Nyström method on N equispaced points on Γ, with the Trapezoidal rule as the quadrature (since the kernel in (5.1) is smooth, the Trapezoidal rule has exponential convergence).

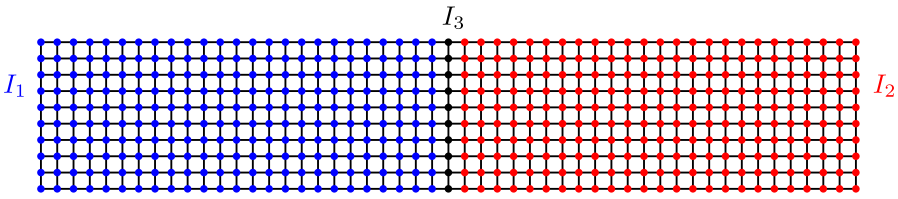


Fig. 11. An example of the grid in the sparse LU example described in Section 5.5. There are $N \times n$ points in the grid, shown for $N = 10, n = 51$.

The fast matrix–vector multiplication is in this case furnished by the recursive skeletonization (RS) procedure of [14]. To avoid spurious effects due to the rank structure inherent in RS, we compute the matrix–vector products at close to double precision accuracy, and with an entirely uncorrelated tree structure.

Results are given in Fig. 13.

Remark 5.1. This problem under consideration here is included to illustrate the asymptotic performance of the proposed method. However, it is artificial in two ways: (1) There is no actual need to use more than a couple of hundred points to resolve (5.1) numerically to double precision accuracy. (2) In this case, we have alternate means of computing a rank structured representation of the matrix. Subsequent examples illustrate more realistic use cases.

5.3. Operator multiplication

We next investigate how the proposed technique performs on a matrix matrix multiplication problem. Specifically, we determine the Neumann-to-Dirichlet operator T for the contour shown in Fig. 10 using the well known formula

$$T = S\left(\frac{1}{2}I + D^*\right)^{-1},$$

where S is the single layer operator $[Sq](x) = \int_\Gamma -\frac{1}{2\pi}\log|x-y|\,q(y)\,ds(y)$, and where D^* is the adjoint of the double-layer operator $[D^*q](x) = \int_\Gamma \frac{n(x)\cdot(x-y)}{2\pi|x-y|^2}\,q(y)\,ds(y)$. The operators S and D are again discretized using a Nyström method on equispaced points (with sixth order Kapur–Rokhlin [31] corrections to handle the singularity in S), resulting in matrices $\mathbf{S}$ and $\mathbf{D}$. The $\mathbf{S}\left(\frac{1}{2}\mathbf{I}+\mathbf{D}^*\right)^{-1}$ is again applied using the recursive skeletonization procedure of [14].

Results are given in Fig. 12.

5.4. Fast multipole method

We consider a kernel matrix representing N-body Laplace interactions in three dimensions, where the interaction kernel is defined by

$$\mathcal{K}(x,y) = \sum_{i\neq j}\frac{c_j}{\|x_i - x_j\|}$$

for sets of N points $\{x_i\}$ and charges $\{c_i\}$. To simulate a problem with low-dimensional structure, we distribute the points uniformly at random on the surface of the unit sphere. We use an implementation of the fast multipole method included in the Flatiron Institute Fast Multipole Libraries [32] to efficiently apply the matrix to vectors. The operator is computed to a target accuracy of $\varepsilon = 10^{-4}$.

Results are given in Fig. 14.

5.5. Frontal matrices in nested dissection

Our next example is a simple model problem that illustrates the behavior of the proposed method in the context of sparse direct solvers. The idea here is to use rank structure to compress the increasingly large Schur complements that arise in the LU factorization of a sparse matrix arising from the finite element or finite difference discretization of an elliptic PDE, cf. [19, Ch. 21]. As a model problem, we consider a $51N \times 51N$ matrix $\mathbf{C}$ that encodes the stiffness matrix for the standard five-point stencil finite difference approximation to the Poisson equation on a rectangle using a grid with $N \times 51$ nodes. We partition the grid into three sets $\{1,2,3\}$, as shown in Fig. 11, and then tessellate $\mathbf{C}$ accordingly,

$$\mathbf{C} = \begin{bmatrix} \mathbf{C}_{11} & \mathbf{0} & \mathbf{C}_{13} \\ \mathbf{0} & \mathbf{C}_{22} & \mathbf{C}_{23} \\ \mathbf{C}_{31} & \mathbf{C}_{32} & \mathbf{C}_{33} \end{bmatrix},$$

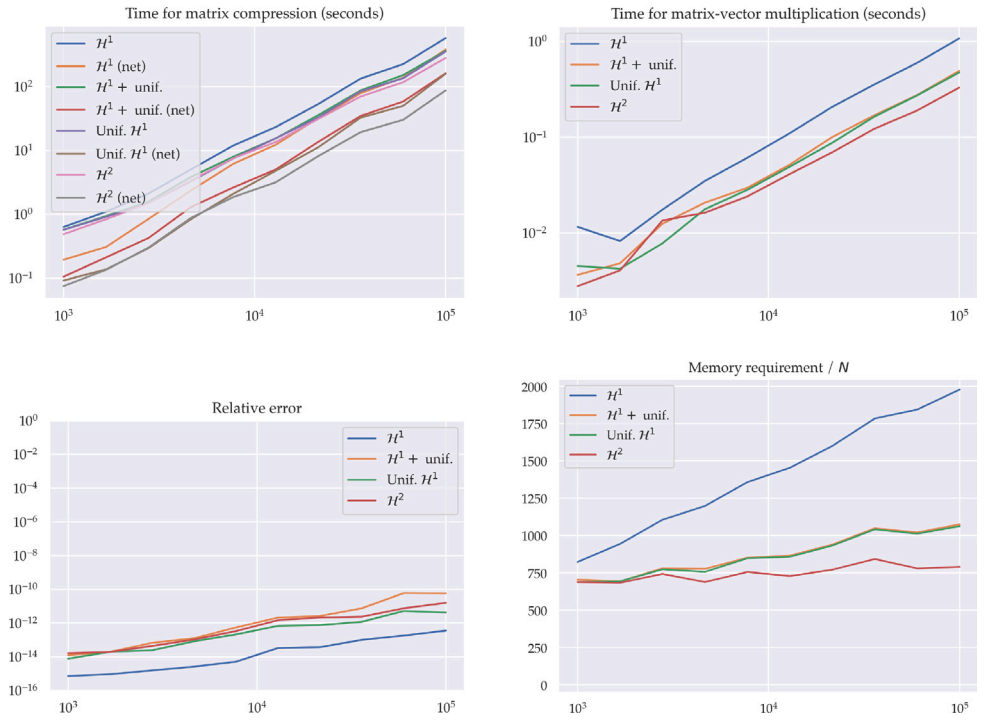


Fig. 12. Results from applying peeling algorithms to the Neumann-to-Dirichlet operator. Here $r = 20$ and $m = 200$, and the horizontal axes represent the problem size N.

where $\mathbf{C}_{11}$ and $\mathbf{C}_{22}$ are $25N \times 25N$, and $\mathbf{C}_{33}$ is $N \times N$. The matrix we seek to compress is the $N \times N$ Schur complement

$$\mathbf{A} = \mathbf{C}_{33} - \mathbf{C}_{31}\mathbf{C}_{11}^{-1}\mathbf{C}_{13} - \mathbf{C}_{32}\mathbf{C}_{22}^{-1}\mathbf{C}_{23}.$$

In our example, we apply $\mathbf{A}$ to a vector by calling standard sparse direct solvers for the left and the right subdomains, respectively. We use sparse matrix routines that are provided by MATLAB and that rely on UMFPACK [33] for the sparse solves.

Results are given in Fig. 15.

5.6. Summary of observations

- The numerical results support our claims of quasilinear computational complexity for all steps of the algorithms presented.
- The unif. $\mathcal{H}^1$ compression scheme consistently outperforms the $\mathcal{H}^1$ + unif. scheme. The advantage appears not only in shorter compression times, but also in higher accuracy and lower storage requirements.
- The approximations achieve high accuracy in every case, with the exception of the 3D FMM, for which the accuracy matches or exceeds the accuracy of the operator.
- The count of floating point numbers per degree of freedom remains constant or grows very slowly for the $\mathcal{H}^2$ format, reflecting linear asymptotic storage cost, and it grows logarithmically for the other formats, reflecting quasilinear asymptotic storage costs.
- In every case, the majority of the compression time is spent in the black-box matrix–vector multiplication routines, highlighting the importance of minimizing the number of matrix–vector products.

6. Conclusions

This paper presents algorithms for randomized compression of rank-structured matrices. The algorithms only access the matrix via black-box matrix–vector multiplication routines. We formulate a graph coloring problem to design sets of test matrices that are tailored to the given matrix and to minimize the number of matrix–vector multiplications required. Numerical experiments indicate that the algorithms are accurate and much more efficient than prior works, particularly when the underlying geometry exhibits low-dimensional structure.

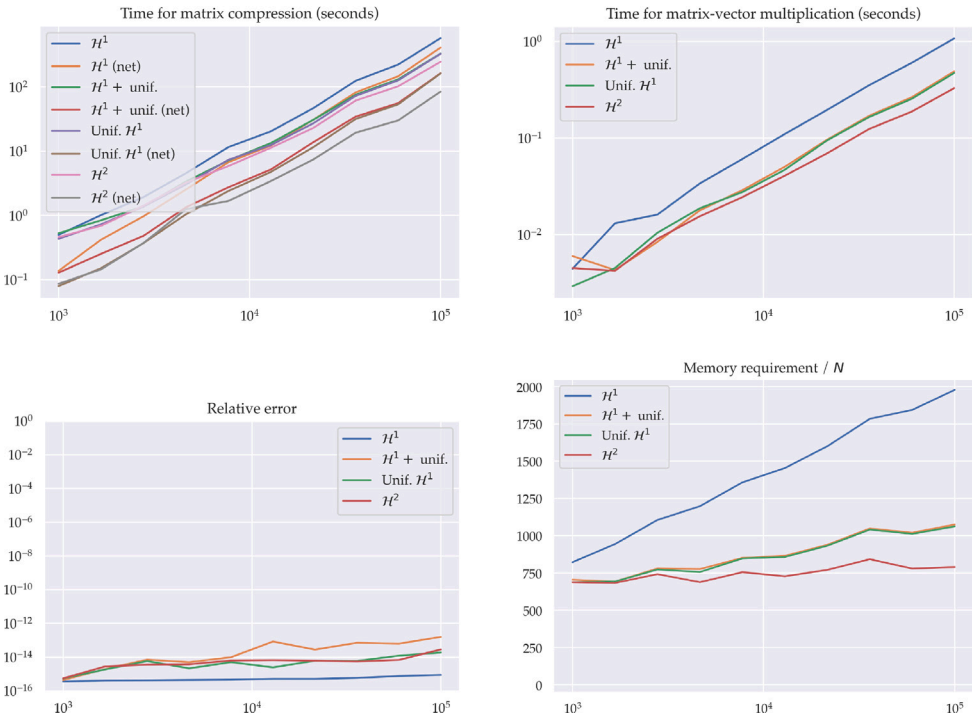


Fig. 13. Results from applying peeling algorithms to a double layer potential on a simple contour in the plane. Here $r = 20$ and $m = 200$, and the horizontal axes represent the problem size N.

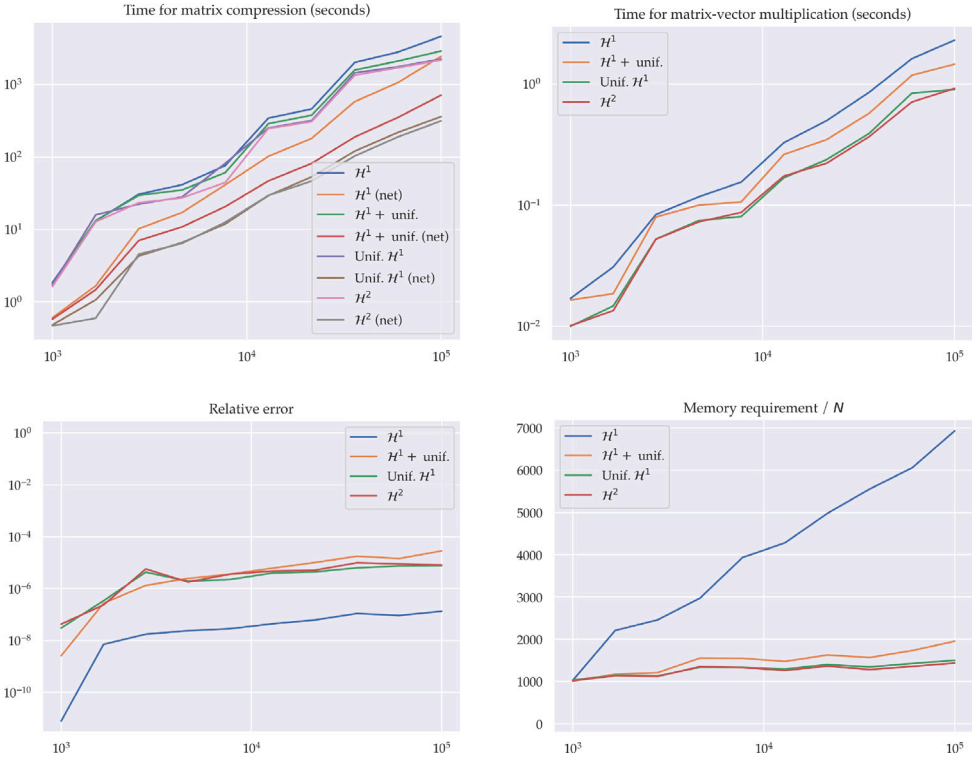


Fig. 14. Results from applying peeling algorithms to the 3D fast multipole method operator. Here $r = 20$ and $m = 50$, and the horizontal axes represent the problem size N.

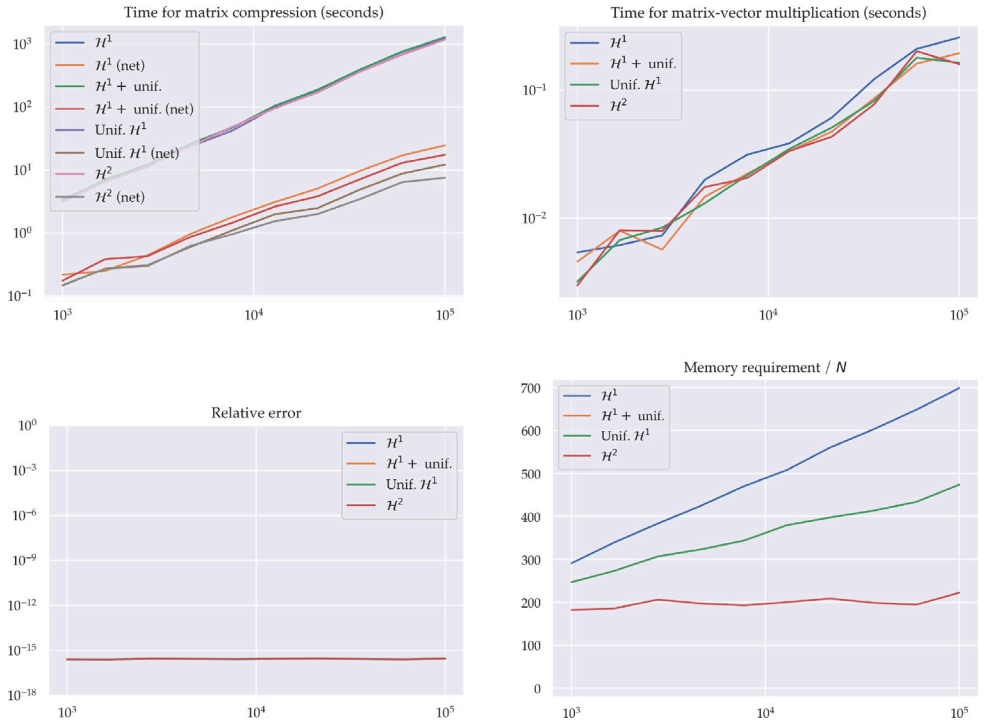


Fig. 15. Results from applying peeling algorithms to frontal matrices in the nested dissection algorithm. Here $r = 10$ and $m = 50$, and the horizontal axes represent the problem size N.

Data availability

No data was used for the research described in the article.

Acknowledgments

The work reported was supported by the Office of Naval Research, United States (N00014-18-1-2354), by the National Science Foundation, United States (DMS-1952735 and DMS-2012606), and by the Department of Energy ASCR (DE-SC0022251).

References

[1] J. Barnes, P. Hut, A hierarchical $o(n \log n)$ force-calculation algorithm, Nature 324 (1986).
[2] L. Greengard, V. Rokhlin, A fast algorithm for particle simulations, J. Comput. Phys. 73 (1987) 325–348.
[3] W. Hackbusch, A sparse matrix arithmetic based on h-matrices. part i: Introduction to h-matrices, Computing 62 (1999) 89–108.
[4] W. Hackbusch, B. Khoromskij, S. Sauter, On $\mathcal{H}^2$-Matrices, in: Lectures on Applied Mathematics, Springer Berlin, 2002, pp. 9–29.
[5] M. Bebendorf, Hierarchical Matrices, Springer, 2008.
[6] S. Börm, Efficient Numerical Methods for Non-Local Operators, in: EMS Tracts in Mathematics, vol. 14, European Mathematical Society (EMS), Zürich, 2010, $\mathcal{H}^2$-matrix compression, algorithms and analysis.
[7] S. Ambikasaran, E. Darve, An o(n log n) fast direct solver for partial hierarchically semi-separable matrices, J. Sci. Comput. 57 (2013) 477–501.
[8] P.-G. Martinsson, A fast direct solver for a class of elliptic partial differential equations, J. Sci. Comput. 38 (2009) 316–330.
[9] S. Chandrasekaran, P. Dewilde, M. Gu, W. Lyons, T. Pals, A fast solver for hss representations via sparse matrices, SIAM J. Matrix Anal. Appl. 29 (2007) 67–81.
[10] S. Chandrasekaran, M. Gu, T. Pals, A fast ulv decomposition solver for hierarchically semiseparable representations, SIAM J. Matrix Anal. Appl. 28 (2006) 603–622.
[11] A. Gillman, P.M. Young, P.-G. Martinsson, A direct solver with o (n) complexity for integral equations on one-dimensional domains, Front. Math. China 7 (2012) 217–247.
[12] J. Xia, S. Chandrasekaran, M. Gu, X.S. Li, Superfast multifrontal method for large structured linear systems of equations, SIAM J. Matrix Anal. Appl. 31 (2010) 1382–1411.
[13] K.L. Ho, L. Greengard, A fast direct solver for structured linear systems by recursive skeletonization, SIAM J. Sci. Comput. 34 (2012) A2507–A2532.
[14] P. Martinsson, V. Rokhlin, A fast direct solver for boundary integral equations in two dimensions, J. Comput. Phys. 205 (2005) 1–23.
[15] V. Minden, K.L. Ho, A. Damle, L. Ying, A recursive skeletonization factorization based on strong admissibility, Multiscale Model. Simul. 15 (2017) 768–796.
[16] J. Xia, Y. Xi, M. Gu, A superfast structured solver for toeplitz linear systems via randomized sampling, SIAM J. Matrix Anal. Appl. 33 (2012) 837–858.
[17] P.R. Amestoy, A. Buttari, J.-Y. L'Excellent, T. Mary, Performance and scalability of the block low-rank multifrontal factorization on multicore architectures, ACM Trans. Math. Softw. 45 (2019) 2.

[18] P. Ghysels, X. Li, F. Rouet, S. Williams, A. Napov, An efficient multicore implementation of a novel hss-structured multifrontal solver using randomized sampling, SIAM J. Sci. Comput. 38 (2016) S358–S384, http://dx.doi.org/10.1137/15M1010117.
[19] P.-G. Martinsson, Fast Direct Solvers for Elliptic PDEs, in: CBMS-NSF Conference Series, vol. CB96, SIAM, 2019, http://dx.doi.org/10.1137/1.9781611976045.
[20] L. Lin, J. Lu, L. Ying, Fast construction of hierarchical matrix representation from matrix–vector multiplication, J. Comput. Phys. 230 (2011) 4071–4087.
[21] P. Martinsson, Rapid factorization of structured matrices via randomized sampling, 2008, arXiv:0806.2339.
[22] P.-G. Martinsson, A fast randomized algorithm for computing a hierarchically semiseparable representation of a matrix, SIAM J. Matrix Anal. Appl. 32 (2011) 1251–1274.
[23] P.-G. Martinsson, Compressing rank-structured matrices via randomized sampling, SIAM J. Sci. Comput. 38 (2016) A1959–A1986.
[24] J. Levitt, P.-G. Martinsson, Linear-complexity black-box randomized compression of rank-structured matrices, 2022, arXiv preprint arXiv:2205.02990.
[25] F. Schäfer, H. Owhadi, Sparse recovery of elliptic solvers from matrix–vector products, 2021, https://arxiv.org/abs/2110.05351.
[26] G.H. Golub, C.F. Van Loan, Matrix Computations, JHU Press, 2013.
[27] N. Halko, P.-G. Martinsson, J.A. Tropp, Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, SIAM Rev. 53 (2011) 217–288.
[28] J.A. Tropp, A. Yurtsever, M. Udell, V. Cevher, Practical sketching algorithms for low-rank matrix approximation, SIAM J. Matrix Anal. Appl. 38 (2017) 1454–1485.
[29] P.-G. Martinsson, J.A. Tropp, Randomized numerical linear algebra: Foundations and algorithms, Acta Numer. 29 (2020) 403–572.
[30] D. Brélaz, New methods to color the vertices of a graph, Commun. ACM 22 (1979) 251–256.
[31] S. Kapur, V. Rokhlin, High-order corrected trapezoidal quadrature rules for singular functions, SIAM J. Numer. Anal. 34 (1997) 1331–1356.
[32] T. Askham, Z. Gimbutas, L. Greengard, L. Lu, J. Magland, D. Malhotra, M. O'Neil, M. Rachh, V. Rokhlin, F. Vico, Flatiron institute fast multipole libraries. https://github.com/flatironinstitute/FMM3D.
[33] T.A. Davis, Algorithm 832: Umfpack v4. 3—an unsymmetric-pattern multifrontal method, ACM Trans. Math. Softw. 30 (2004) 196–199.